



Проект спринта: Нелинейные модели против сочинской погоды

Компания **BikeSochi**, оператор городского велопроката в Сочи, обратилась за помощью в улучшении системы прогнозирования почасового спроса на велосипеды.

До этого аналитики компании использовали простую линейную регрессию — одну из самых базовых моделей машинного обучения.

Однако её прогнозы часто ошибались:



«Модель видела, что температура выросла — и сразу предсказывала высокий спрос. Но не понимала, что +28 °C под палящим солнцем и +28 °C после дождя — это две совсем разные ситуации.»

Погода в Сочи меняется быстро, и влияние факторов — нелинейное: температура, влажность, солнечная радиация, осадки влияют на поведение людей не по простым прямым зависимостям.

В рамках этого проекта вы выступаете в роли аналитиков компании **BikeSochi**, которые **должны предложить новую, более гибкую модель прогнозирования**.

В этом проекте вы:

1. Изучите предоставленную компанией baseline-модель (линейную регрессию);
2. Обучите новые модели — K-ближайших соседей (KNN) и решающее дерево (Decision Tree);
3. Проведёте подбор гиперпараметров с помощью библиотеки Optuna;
4. Сравните результаты всех подходов, выберете наилучший и аргументируете свой выбор.

Дополнительно: Реализуете собственный класс-трансформер и интегрируете его в пайплайн.

К концу проекта у вас будет:

- улучшенная модель прогноза почасового спроса на велосипеды в Сочи;
 - отчёт с метриками и сравнением подходов;
 - сохранённый пайплайн лучшей модели.
-

Часть 1: Базовая модель

В этой части вы познакомитесь с тем, как работает baseline-модель, которую использовала компания BikeSochi до внедрения улучшенной версии.

Компания предоставила вам:

- `baseline_linear_regression_pipeline.pkl` — готовый обученный пайплайн (без исходного кода);
- краткое описание того, как он был устроен;
- тренировочную и тестовую выборки, которые можно использовать для оценки модели.

Это значит, что вам не нужно обучать эту модель заново — вы просто загружаете её и проверяете качество (например, на метриках RMSE, MAE, R^2).

Совет:

1. Убедитесь, что у вас есть `baseline_linear_regression_pipeline.pkl` и `train_data.csv` и `test_data.csv`.
2. Разделите тестовый набор на признаки (X) и целевую переменную Rented Bike Count.
3. Загрузите .pkl файл — это готовый пайплайн, который сам обрабатывает данные и делает предсказания. Модель автоматически применяет трансформации и возвращает прогнозы.
4. Посчитайте RMSE, MAE и R^2 , эти результаты нужны для оценки ваших улучшенных моделей в дальнейшей работе.

Часть 2: Улучшение модели — KNN и Decision Tree

Ваша задача — предложить более гибкую модель прогноза спроса на велосипеды, которая учитывает нюансы сочинской погоды и поведение клиентов.

Вы будете экспериментировать с К-ближайшими соседями (KNN) и решающим деревом (Decision Tree), использовать подбор гиперпараметров с Optuna и внедрять кастомные трансформеры для новых признаков.

Шаг 1: Изучение постановки задачи:

Проведите EDA (Exploratory Data Analysis):

1. Посмотрите на распределение целевой переменной Rented Bike Count. Есть ли выбросы или сильные сезонные колебания?
2. Постройте графики зависимости спроса от температуры, осадков, солнечной радиации.
3. Сравните спрос в разные сезоны и праздничные дни.
4. Проверьте корреляции между признаками и целевой переменной.

Совет:

Не нужно сразу всё усложнять. Начните с базовых графиков и описательной статистики.

Шаг 2: Разделение данных на тренировочную и валидационную выборки

У вас есть доступ к тренировочной и тестовой выборкам.

Тестовый набор используется для финальной оценки модели после тренировки и подбора гиперпараметров.

Вы тренируетесь на тренировочных данных и оцениваете качество через валидацию.

Шаг 3: Обучение новых моделей

Теперь пора «проверить, насколько далеко уедут модели» — KNN и Decision Tree могут уловить нелинейные зависимости, недоступные линейной регрессии.

Что нужно сделать:

1. Подготовьте пайплайн для каждой модели:
 - Предобработка данных.
 - Модель (KNN или Decision Tree).
2. Настройте базовые параметры моделей (например, `n_neighbors` для KNN, `max_depth` для дерева).

Совет:

Начинайте с базовых параметров, чтобы убедиться, что пайплайн работает. Оптимизацию параметров делаем на следующем шаге.

Шаг 4: Подбор гиперпараметров с Optuna

Компания хочет точную модель. Optuna поможет найти лучшие параметры для KNN и дерева, чтобы снизить ошибки прогноза.

Что нужно сделать:

1. Определите функцию цели для Optuna.
2. Настройте диапазоны гиперпараметров.
3. Запустите оптимизацию и сохраните лучшие параметры.

Совет:

Не бойтесь экспериментировать с небольшими диапазонами сначала, а потом расширять, если модель «не уловила» зависимости.

Шаг 5: Кросс-валидация новых моделей

Что нужно сделать:

1. Проведите кросс-валидацию для KNN и Decision Tree с оптимальными гиперпараметрами.
2. Сравните метрики с baseline-моделью.
3. Определите, какая модель показывает лучшие результаты на тренировочной выборке.

Совет:

Используйте визуализации (bar plot, box plot), чтобы увидеть разброс метрик и стабильность моделей.

Шаг 6: Составление отчёта по моделям

Что нужно сделать:

1. Составьте таблицу с метриками: baseline, KNN, Decision Tree.
2. Добавьте визуализацию, если необходимо: bar plot или box plot по RMSE/MAE.

Подготовьте выводы:

1. Какая модель лучше справляется с прогнозом?
2. Какие признаки, по вашему мнению, особенно важны?

Совет:

Старайтесь объяснить результаты в бизнес-контексте: «Эта модель лучше реагирует на дождь», «Температура и влажность сильно влияют на спрос в пиковые часы».

Шаг 7: Сохранение модели и отчёта

Что нужно сделать:

1. Выберите финальную, наилучшую модель и оцените её качество на тестовой выборке, чтобы понять, насколько она прогнозирует реальные данные.
2. Подготовьте ноутбук с кодом и комментариями: включите результаты всех экспериментов, метрики моделей, визуализации, а также обоснование выбора финальной модели.

Совет:

Документируйте каждый шаг: почему выбраны те или иные гиперпараметры и подходы. В реальной бизнес-задаче это помогает коллегам и руководству понимать решения и доверять модели.

Опционально: Реализация кастомного трансформера

Простые признаки вроде температуры или влажности не всегда отражают реальную ситуацию. Чтобы модель могла лучше прогнозировать спрос на велосипеды, можно

создавать новые признаки, которые учитывают особенности погоды или взаимодействия факторов.

Что нужно сделать:

1. Реализуйте класс с методами `fit` и `transform`.
2. Вставьте его в пайплайн перед моделью.
3. Проверьте, что трансформер корректно работает с тренировочными данными.

Совет: Если вы решите реализовать трансформер, начинайте с простых комбинаций признаков, чтобы не усложнять модель слишком рано.

Авторское решение

Цель:

Разработать модель машинного обучения для более точного прогнозирования почасового спроса на велосипеды, способную учитывать нелинейные зависимости между погодными условиями и активностью

Задача:

Сравнить качество базовой модели линейной регрессии с нелинейными моделями (KNN и Decision Tree), подобрать гиперпараметры с помощью Optuna, выбрать лучшую модель и сохранить готовый пайплайн для дальнейшего использования.

План работы

- Этап 1. Подготовка среды: загрузка библиотек, фиксация `random_state`, загрузка обучающего и тестового датасетов
- Этап 2. Анализ данных (EDA): изучение структуры данных, типов признаков, проверка пропусков и дубликатов, анализ распределения целевой переменной
- Этап 3 Предобработка: Собираем пайплайн заполняем пропуски отделяем целевую переменную
- Этап 4. Обучение моделей: построение пайплайнов для KNN и Decision Tree
- Этап 5. Подбор гиперпараметров: оптимизация параметров моделей с помощью библиотеки Optuna
- Этап 6. Сравнение моделей: анализ метрик качества, сравнение результатов baseline и новых моделей, выбор лучшей модели.

- Этап 7. Финальная модель: обучение лучшей модели, сохранение готового пайплайна с помощью `joblib`.
- Этап 8. Отчёт: описание полученных результатов, сравнение моделей и обоснование выбора лучшего решения.

Этап 1 Подготовка среды: загрузка данных, фиксация `random_state`, установка библиотек.

- Установим и настроим библиотеки
- Зафиксируем `random_state` и параметры отображения
- Загрузим данные из CSV-файлов

Установим и настроим библиотеки

```
In [1]: %%writefile requirements.txt
pandas==2.3.3
numpy==1.26.4
matplotlib==3.7.3
seaborn==0.13.2
phik==0.12.5
scikit-learn==1.6.1
optuna==4.8.0
joblib==1.5.3
```

```
Out:
Overwriting requirements.txt
```

```
In [2]: !pip install -r requirements.txt -q
```

```
In [3]: # !pip install phik
# !pip install optuna
```

```
import urllib.request
import numpy as np
import pandas as pd
import phik
import joblib
import optuna
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_validate, KFold, cross_val_predict, cross_val_score
from sklearn.metrics import make_scorer, mean_absolute_error, mean_squared_error, r2_score
from sklearn.inspection import permutation_importance

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler, OrdinalEncoder

from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
```

Зафиксируем `random_state` и параметры отображения

```
In [4]: plt.style.use('default')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

RANDOM_STATE = 42
```

- `plt.style.use('default')` - использование стандартного стиля визуализации matplotlib.
- `plt.rcParams['figure.figsize'] = (10, 6)` - размер графиков.
- `plt.rcParams['font.size'] = 12` - размер шрифта на графиках.
- `RANDOM_STATE = 42` - фиксация генератора случайных чисел для воспроизводимости результатов.

Загрузим данные из CSV-файлов:

- `ds_s14_train_data.csv`
- `ds_s14_test_data.csv`


```
In [5]: try:
        train = pd.read_csv('ds_s14_train_data.csv')
        test = pd.read_csv('ds_s14_test_data.csv')

    except:
        train = pd.read_csv('https://code.s3.yandex.net/datasets/ds_s14_train_data.csv')
        test = pd.read_csv('https://code.s3.yandex.net/datasets/ds_s14_test_data.csv')
```

Проверка выгруженных данных

```
In [6]: datasets = {
        'train': train,
        'test': test
    }

    for name, df in datasets.items():
        print(f'{name}: {df.shape}')
```

```
Out:
train: (7008, 16)
test: (1752, 16)
```

Знакомство и оценка base-line модели компании

```
In [7]: baseline_pipeline= joblib.load('/datasets/baseline_linear_regression_pipeline.pkl')

X_train_base = train.drop(columns='Rented Bike Count')
y_train_base = train['Rented Bike Count']
X_test_base = test.drop(columns='Rented Bike Count')
y_test_base = test['Rented Bike Count']

y_train_pred_base = baseline_pipeline.predict(X_train_base)
y_test_pred_base = baseline_pipeline.predict(X_test_base)

mse_train = mean_squared_error(y_train_base, y_train_pred_base)
mse_test = mean_squared_error(y_test_base, y_test_pred_base)
```

```

metrics_baseline = pd.DataFrame([{'dataset': 'train',
                                  'MAE': mean_absolute_error(y_train_base, y_train
                                                              _pred_base),
                                  'MSE': mse_train,
                                  'RMSE': np.sqrt(mse_train),
                                  'R2': r2_score(y_train_base, y_train_pred_base)},
                                  {'dataset': 'test',
                                   'MAE': mean_absolute_error(y_test_base, y_test_p
                                                              red_base),
                                   'MSE': mse_test,
                                   'RMSE': np.sqrt(mse_test),
                                   'R2': r2_score(y_test_base, y_test_pred_base)}}])

print(metrics_baseline)

```

Out:

	dataset	MAE	MSE	RMSE	R2
0	train	309.078381	170177.993183	412.526355	0.592544
1	test	312.599334	169384.999132	411.564089	0.586072

Вывод по базовой модели компании

Baseline модель показала близкие метрики на train и test. Модель не переобучена: одинаково работает на обучающих, и на новых данных. Но R^2 0.59 означает, что модель объясняет лишь 59% спроса, а средняя ошибка прогноза (~310 велосипедов) сопоставима с медианным значением целевой переменной (~500). Это подтверждает исходную гипотезу заказчика: линейная регрессия не способна улавливать нелинейные зависимости спроса от температуры, влажности и других погодных факторов, выявленные на этапе EDA — отсюда и необходимость перехода на более гибкие модели (KNN, Decision Tree).

Этап 2 Анализ данных (EDA): изучаем признаки, ищем пропуски, дубликаты, выбросы, проверяем корреляции между признаками.

Первичный анализ выгруженных данных

```

In [8]: def datasets_info(datasets):
        for name, df in datasets.items():

```

```

print(name)

for col in df.columns:
    print(f"\t{col}"
          f"\n\t\tтип: {df[col].dtype}"
          f"\n\t\tвсего строк: {len(df[col])}"
          f"\n\t\tпропусков: {df[col].isna().sum()}"
          f"\n\t\tпримеры возможных значений: {list(df[col].dropna().unique()[:5])}")
    print()
datasets_info(datasets)

```

Out:

train

Temperature

```

тип: float64
всего строк: 7008
пропусков: 0
примеры возможных значений: [20.3, 25.4, -6.9, -5.2, 23.4]

```

Humidity(%)

```

тип: float64
всего строк: 7008
пропусков: 250
примеры возможных значений: [35.0, 55.0, 39.0, 37.0, 34.0]

```

Wind speed (m/s)

```

тип: float64
всего строк: 7008
пропусков: 210
примеры возможных значений: [2.4, 3.2, 1.6, 2.2, 2.1]

```

Visibility (10m)

```

тип: float64
всего строк: 7008
пропусков: 259
примеры возможных значений: [2000.0, 1308.0, 1911.0, 1553.0, 118.0]

```

Dew point temperature

```

тип: float64
всего строк: 7008
пропусков: 0
примеры возможных значений: [4.3, 15.6, -18.5, -17.6, 6.6]

```

Solar Radiation (MJ/m2)

```

тип: float64
всего строк: 7008
пропусков: 210
примеры возможных значений: [0.46, 0.15, 0.0, 2.84, 3.24]

```

Rainfall(mm)

```

тип: float64
всего строк: 7008
пропусков: 262

```

примеры возможных значений: [0.0, 7.3, 1.5, 0.5, 4.9]

Snowfall (cm)

 тип: float64

 всего строк: 7008

 пропусков: 263

 примеры возможных значений: [0.0, 4.0, 0.5, 0.3, 1.0]

Seasons

 тип: object

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: ['Autumn', 'Winter', 'Summer', 'Spring']

Holiday

 тип: object

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: ['Holiday', 'No Holiday']

Functioning Day

 тип: object

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: ['Yes', 'No']

Time_Period_Evening

 тип: bool

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: [True, False]

Time_Period_Late Evening

 тип: bool

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: [False, True]

Time_Period_Morning

 тип: bool

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: [False, True]

Time_Period_Night

 тип: bool

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: [False, True]

Rented Bike Count

 тип: int64

 всего строк: 7008

 пропусков: 0

 примеры возможных значений: [1237, 2468, 186, 254, 1686]

test

Temperature

 тип: float64

всего строк: 1752
пропусков: 0
примеры возможных значений: [6.7, -5.3, 2.9, 23.1, 23.3]

Humidity(%)
тип: float64
всего строк: 1752
пропусков: 72
примеры возможных значений: [48.0, 90.0, 66.0, 63.0, 43.0]

Wind speed (m/s)
тип: float64
всего строк: 1752
пропусков: 52
примеры возможных значений: [0.8, 1.1, 1.2, 2.6, 1.3]

Visibility (10m)
тип: float64
всего строк: 1752
пропусков: 72
примеры возможных значений: [817.0, 311.0, 173.0, 949.0, 1257.0]

Dew point temperature
тип: float64
всего строк: 1752
пропусков: 0
примеры возможных значений: [-3.5, -6.6, -2.8, 15.6, 10.0]

Solar Radiation (MJ/m2)
тип: float64
всего строк: 1752
пропусков: 52
примеры возможных значений: [0.39, 0.0, 1.68, 1.91, 2.46]

Rainfall(mm)
тип: float64
всего строк: 1752
пропусков: 64
примеры возможных значений: [0.0, 2.5, 0.5, 4.5, 13.5]

Snowfall (cm)
тип: float64
всего строк: 1752
пропусков: 61
примеры возможных значений: [0.0, 2.2, 0.5, 0.8, 1.9]

Seasons
тип: object
всего строк: 1752
пропусков: 0
примеры возможных значений: ['Winter', 'Spring', 'Summer', 'Autumn']

Holiday
тип: object
всего строк: 1752
пропусков: 0
примеры возможных значений: ['No Holiday', 'Holiday']

Functioning Day
тип: object

```
    всего строк: 1752
    пропусков: 0
    примеры возможных значений: ['Yes', 'No']
Time_Period_Evening
    тип: bool
    всего строк: 1752
    пропусков: 0
    примеры возможных значений: [False, True]
Time_Period_Late Evening
    тип: bool
    всего строк: 1752
    пропусков: 0
    примеры возможных значений: [False, True]
Time_Period_Morning
    тип: bool
    всего строк: 1752
    пропусков: 0
    примеры возможных значений: [False, True]
Time_Period_Night
    тип: bool
    всего строк: 1752
    пропусков: 0
    примеры возможных значений: [False, True]
Rented Bike Count
    тип: int64
    всего строк: 1752
    пропусков: 0
    примеры возможных значений: [402, 178, 219, 949, 1005]
```

Вывод

Обучающая и тестовая выборки имеют одинаковый набор признаков. Основная часть признаков описывает погодные условия, также представлены календарные признаки и признаки времени суток.

В данных присутствуют пропуски в числовых признаках. Бинарные признаки `holiday` и `Functioning Day` имеют категориальный тип; для более эффективного хранения их можно было бы представить в виде булевых значений. Признаки `Time_Period_*` являются результатом предварительного One-Hot Encoding. Для более гибкой и воспроизводимой предобработки предпочтительнее хранить исходный категориальный признак и выполнять кодирование внутри `Pipeline`.

Распределение целевой переменной

```

In [9]: def data_review(df, col):
        data = df[col].dropna()
        print(data.describe())

        q1 = data.quantile(0.25)
        q3 = data.quantile(0.75)
        iqr = q3 - q1
        lower_bound = q1 - 1.5 * iqr
        upper_bound = q3 + 1.5 * iqr

        outliers = data[(data < lower_bound) | (data > upper_bound)]
        high_outliers = data[data > upper_bound].sort_values(ascending=True)

        print("Выбросы:")
        print("Количество:", len(outliers))
        print("Процент:", round(len(outliers) / len(data) * 100, 2))
        print("Границы:", round(lower_bound, 2), round(upper_bound, 2))
        print("Первые 5 выбросов выше верхней границы:", high_outliers.head(5).values)
        print("5 максимальных значений:", high_outliers.tail(5).values)

        sns.histplot(data, kde=True)
        plt.title(f"Hist: {col}")
        plt.show()

        sns.boxplot(x=data)
        plt.title(f"Boxplot: {col}")
        plt.show()

        data_review(train, 'Rented Bike Count')
        data_review(test, 'Rented Bike Count')

```

Out:

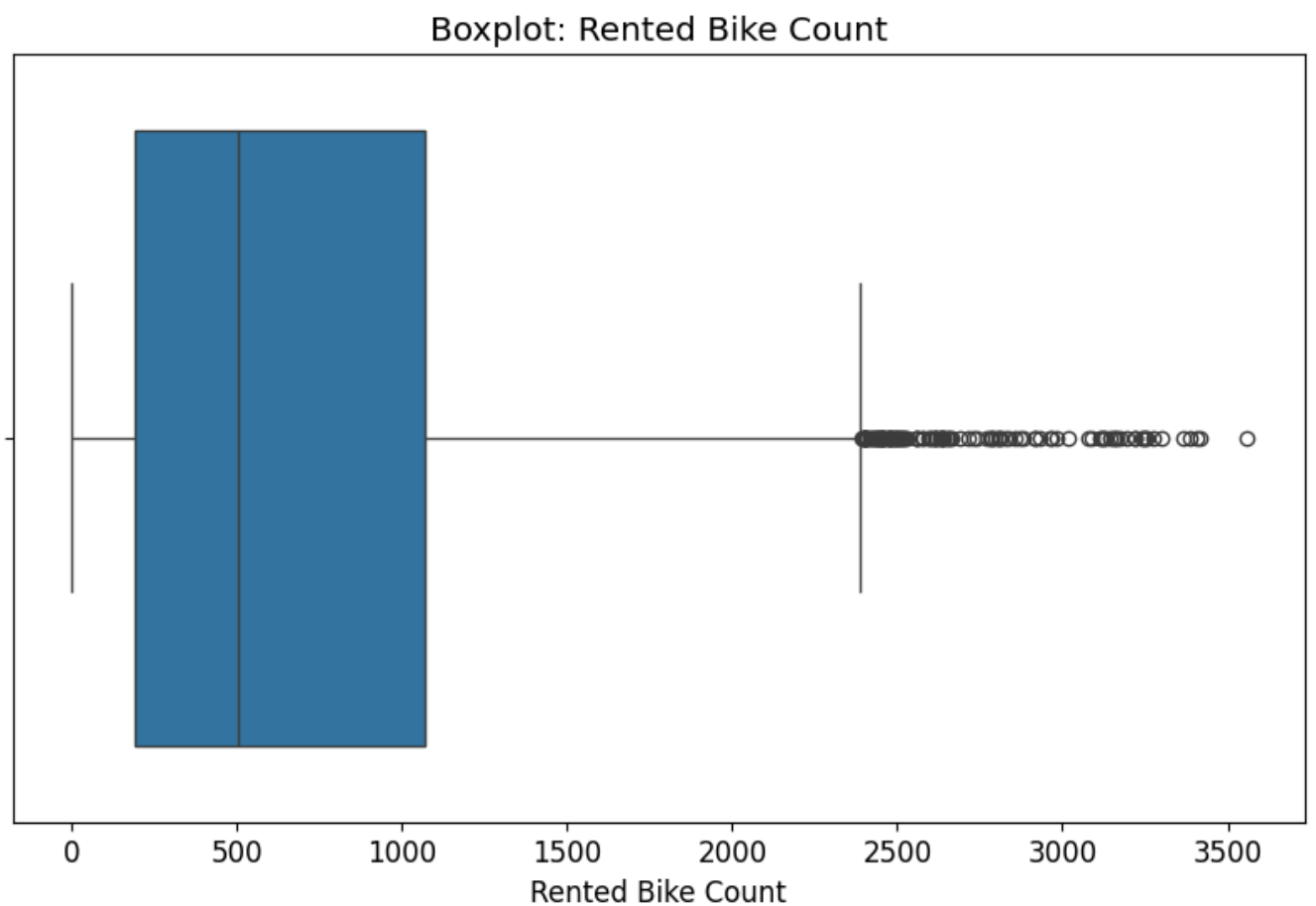
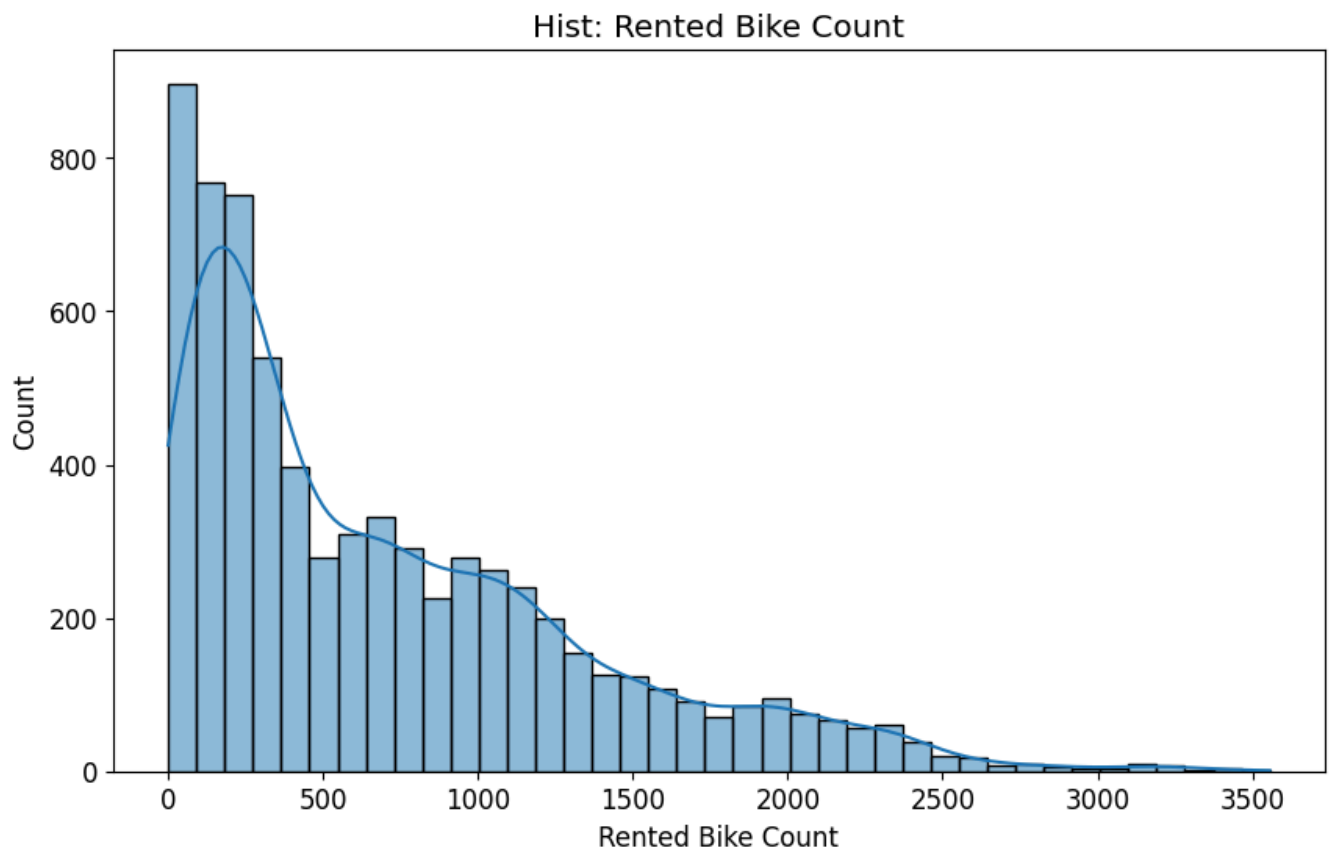
```

count      7008.000000
mean        705.606022
std         646.311790
min          0.000000
25%         190.750000
50%         504.500000
75%        1070.000000
max        3556.000000
Name: Rented Bike Count, dtype: float64
Выбросы:
Количество: 121
Процент: 1.73
Границы: -1128.12 2388.88

```

Первые 5 выбросов выше верхней границы: [2392 2397 2398 2400 2401]

5 максимальных значений: [3365 3384 3404 3418 3556]




```
count    1752.000000
mean      700.586187
std       639.880816
min        0.000000
25%       194.750000
50%       503.500000
75%      1042.000000
max      3380.000000
```

Name: Rented Bike Count, dtype: float64

Выбросы:

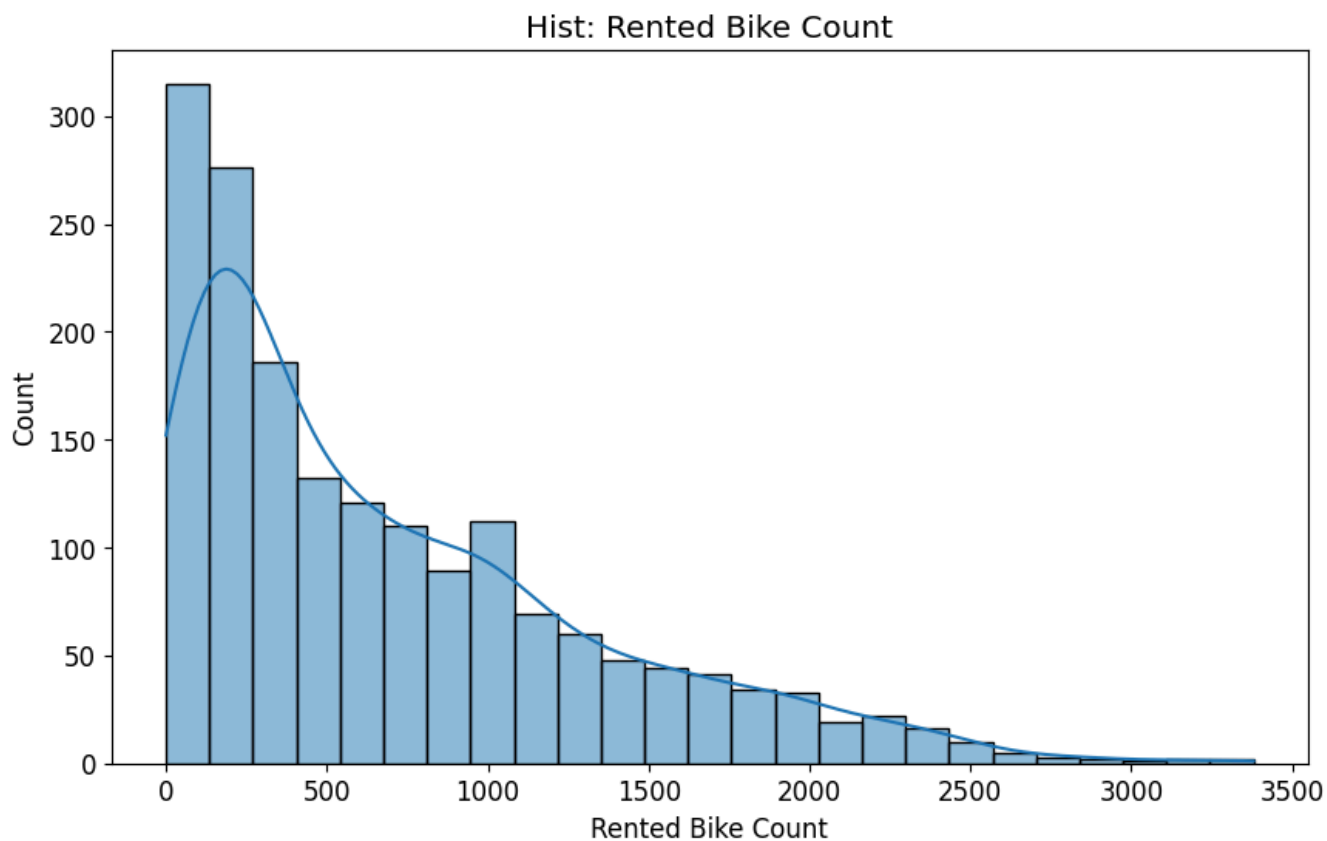
Количество: 39

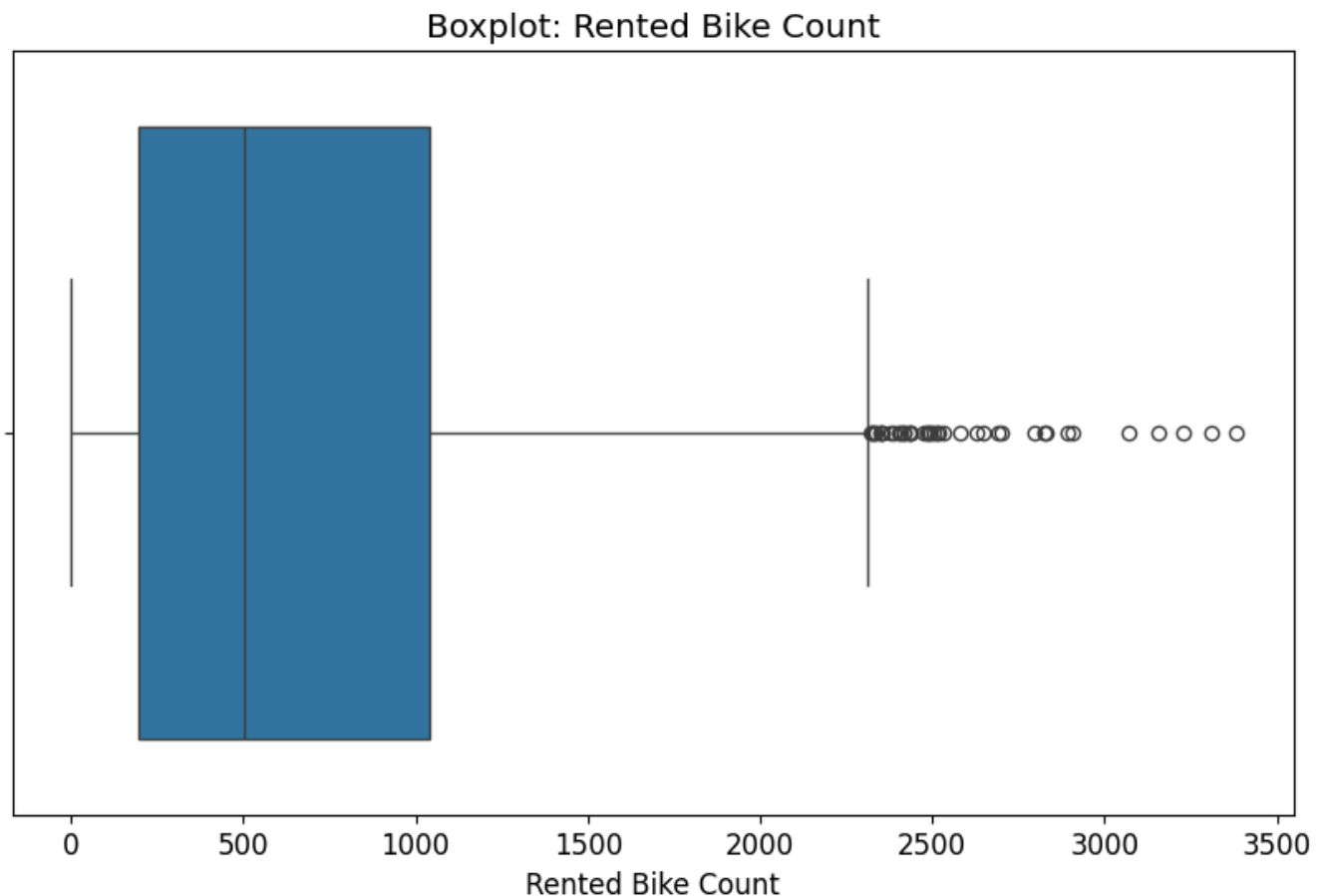
Процент: 2.23

Границы: -1076.12 2312.88

Первые 5 выбросов выше верхней границы: [2322 2326 2330 2337 2348]

5 максимальных значений: [3069 3154 3227 3309 3380]





Вывод

Распределение целевой переменной в обучающей и тестовой выборках совпадает. Среднее значение, медиана, q характеристики и стандартное отклонение отличаются незначительно

В обеих выборках присутствует небольшое количество выбросов, при этом максимальные значения не сильно различаются. Существенных различий в распределении целевой переменной нет

```
In [10]: def missing_report(df):
    total = len(df)
    for col in df.columns:
        print(col)
        print("\t", "пропусков:", df[col].isna().sum())
        print("\t", f"доля: {round(df[col].isna().sum() / total * 100, 2)}%")
    df_clean = df.dropna()
    print("Без удаления пропусков строк:", len(df))
    print("После удаления:", len(df_clean))
    print("Потеря:", len(df) - len(df_clean))
    print("Доля потерь:", (len(df) - len(df_clean)) / len(df))

    print("Обучающая выборка")
    missing_report(train)
```

```
print("\nТестовая выборка")  
missing_report(test)
```

Out:

```
Обучающая выборка  
Temperature  
    пропусков: 0  
    доля: 0.0%  
Humidity(%)  
    пропусков: 250  
    доля: 3.57%  
Wind speed (m/s)  
    пропусков: 210  
    доля: 3.0%  
Visibility (10m)  
    пропусков: 259  
    доля: 3.7%  
Dew point temperature  
    пропусков: 0  
    доля: 0.0%  
Solar Radiation (MJ/m2)  
    пропусков: 210  
    доля: 3.0%  
Rainfall(mm)  
    пропусков: 262  
    доля: 3.74%  
Snowfall (cm)  
    пропусков: 263  
    доля: 3.75%  
Seasons  
    пропусков: 0  
    доля: 0.0%  
Holiday  
    пропусков: 0  
    доля: 0.0%  
Functioning Day  
    пропусков: 0  
    доля: 0.0%  
Time_Period_Evening  
    пропусков: 0  
    доля: 0.0%  
Time_Period_Late Evening  
    пропусков: 0  
    доля: 0.0%  
Time_Period_Morning  
    пропусков: 0  
    доля: 0.0%  
Time_Period_Night
```

пропусков: 0

доля: 0.0%

Rented Bike Count

пропусков: 0

доля: 0.0%

Без удаления пропусков строк: 7008

После удаления: 5674

Потеря: 1334

Доля потерь: 0.1903538812785388

Тестовая выборка

Temperature

пропусков: 0

доля: 0.0%

Humidity(%)

пропусков: 72

доля: 4.11%

Wind speed (m/s)

пропусков: 52

доля: 2.97%

Visibility (10m)

пропусков: 72

доля: 4.11%

Dew point temperature

пропусков: 0

доля: 0.0%

Solar Radiation (MJ/m2)

пропусков: 52

доля: 2.97%

Rainfall(mm)

пропусков: 64

доля: 3.65%

Snowfall (cm)

пропусков: 61

доля: 3.48%

Seasons

пропусков: 0

доля: 0.0%

Holiday

пропусков: 0

доля: 0.0%

Functioning Day

пропусков: 0

доля: 0.0%

Time_Period_Evening

пропусков: 0

доля: 0.0%

Time_Period_Late Evening

пропусков: 0

доля: 0.0%

```
Time_Period_Morning
    пропусков: 0
    доля: 0.0%
Time_Period_Night
    пропусков: 0
    доля: 0.0%
Rented Bike Count
    пропусков: 0
    доля: 0.0%
Без удаления пропусков строк: 1752
После удаления: 1419
Потеря: 333
Доля потерь: 0.19006849315068494
```

Вывод:

Удаление строк с пропусками приводит к значительной потере данных (19%), поэтому не имеет смысла. Пропуски равномерно распределены в обоих датасетах. Оптимально использовать `SimpleImputer` для заполнения пропусков.

Обработка дубликатов

```
In [11]: def duplicates_report(df, id_col=None):
        full_dup = df.duplicated().sum()
        print("Полные дубликаты строк:", full_dup)

        if id_col is not None and id_col in df.columns:
            id_dup = df.duplicated(subset=[id_col]).sum()
            print("Дубликаты по id:", id_dup)
            no_id = df.drop(columns=[id_col])
            no_id_dup = no_id.duplicated().sum()
            print("Дубликаты без id:", no_id_dup)

        print("Обучающая выборка")
        duplicates_report(train)
        print("\nТестовая выборка")
        duplicates_report(test)
```

Out:

```
Обучающая выборка
Полные дубликаты строк: 0
```

Тестовая выборка
Полные дубликаты строк: 0

дубликатов не обнаружено

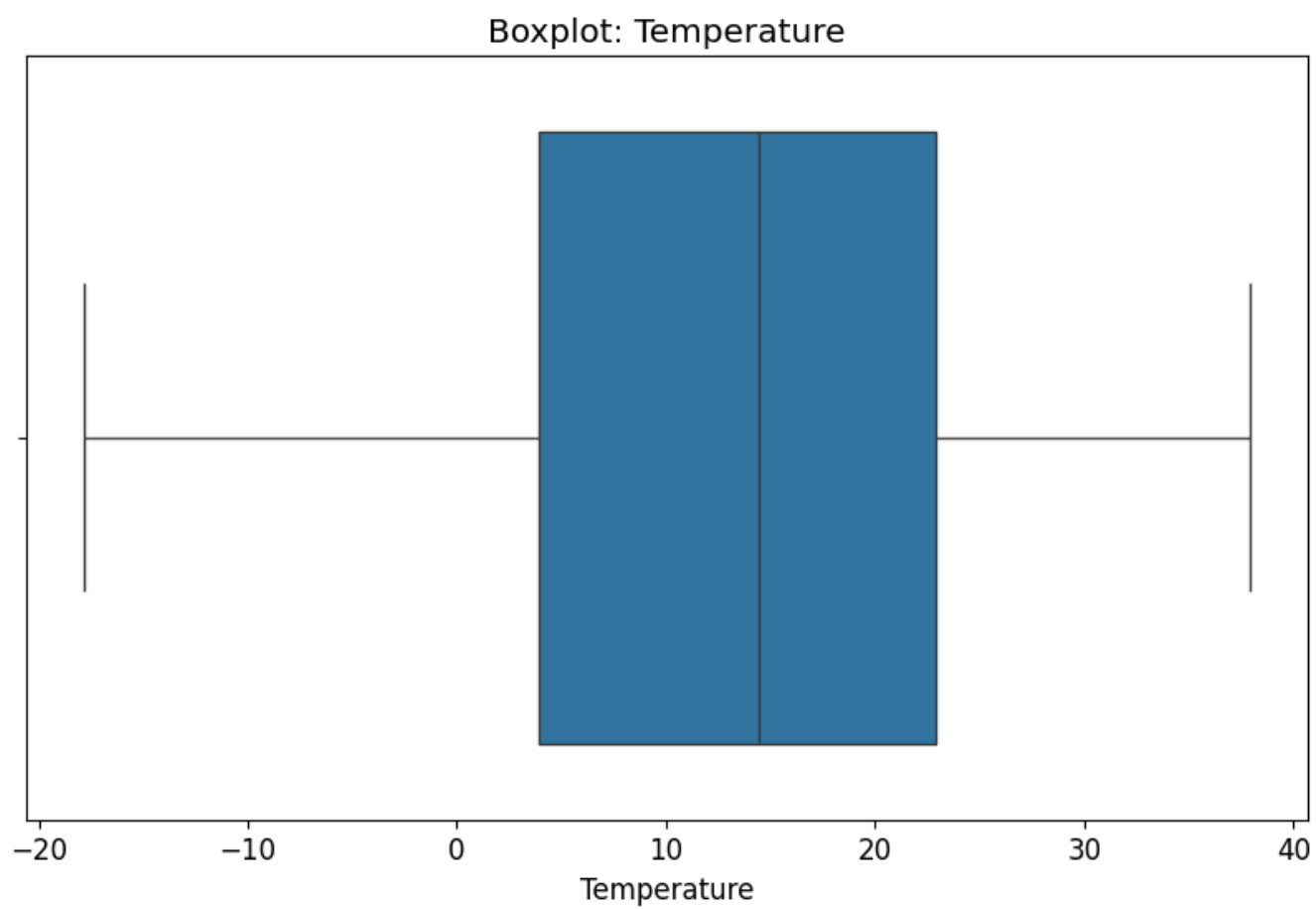
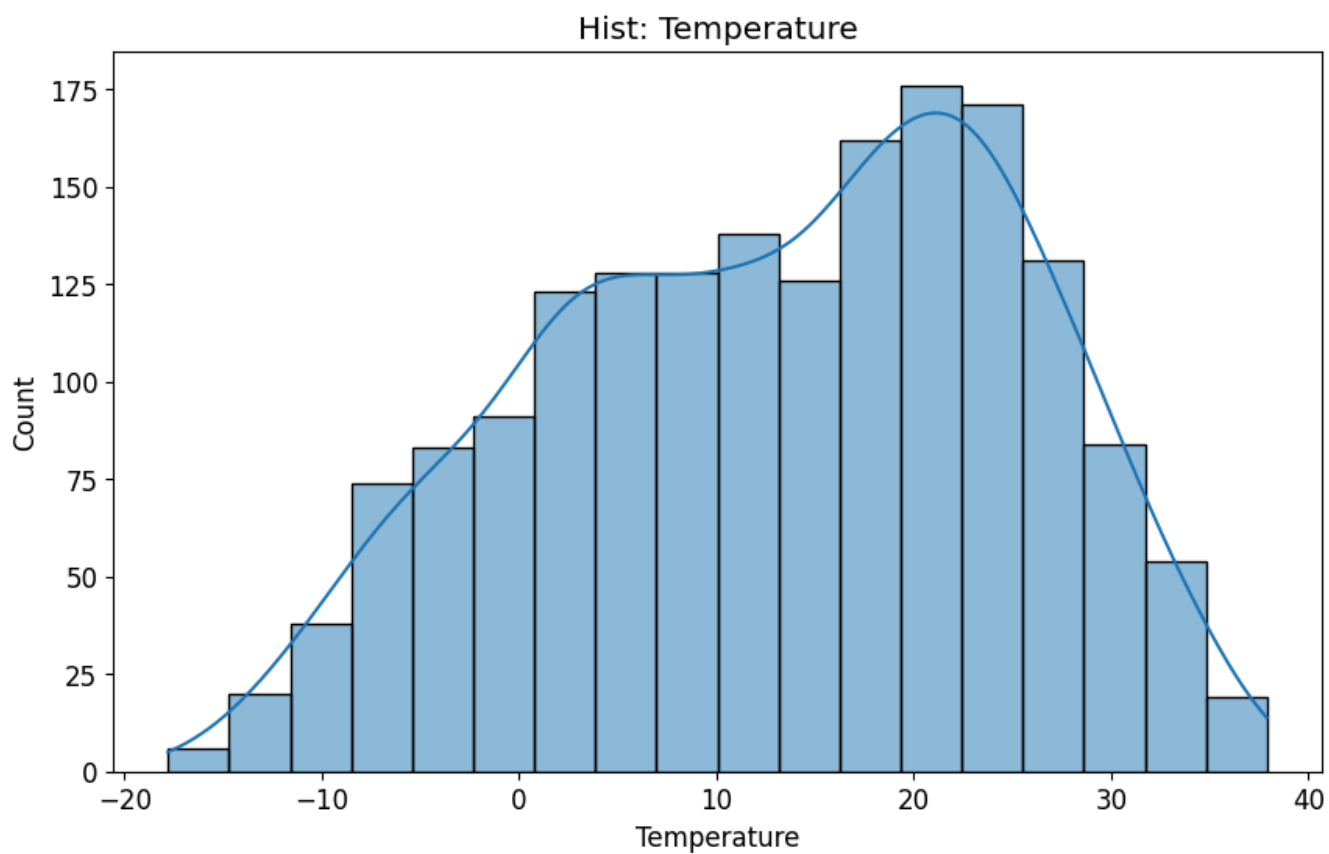
Анализ выбросов и распределения

```
In [12]: num_cols = df.select_dtypes(include=["int64", "float64"]).columns
num_cols = num_cols.drop("Rented Bike Count", errors="ignore")
```

```
for col in num_cols:
    print(col)
    data_review(df, col)
```

Out:

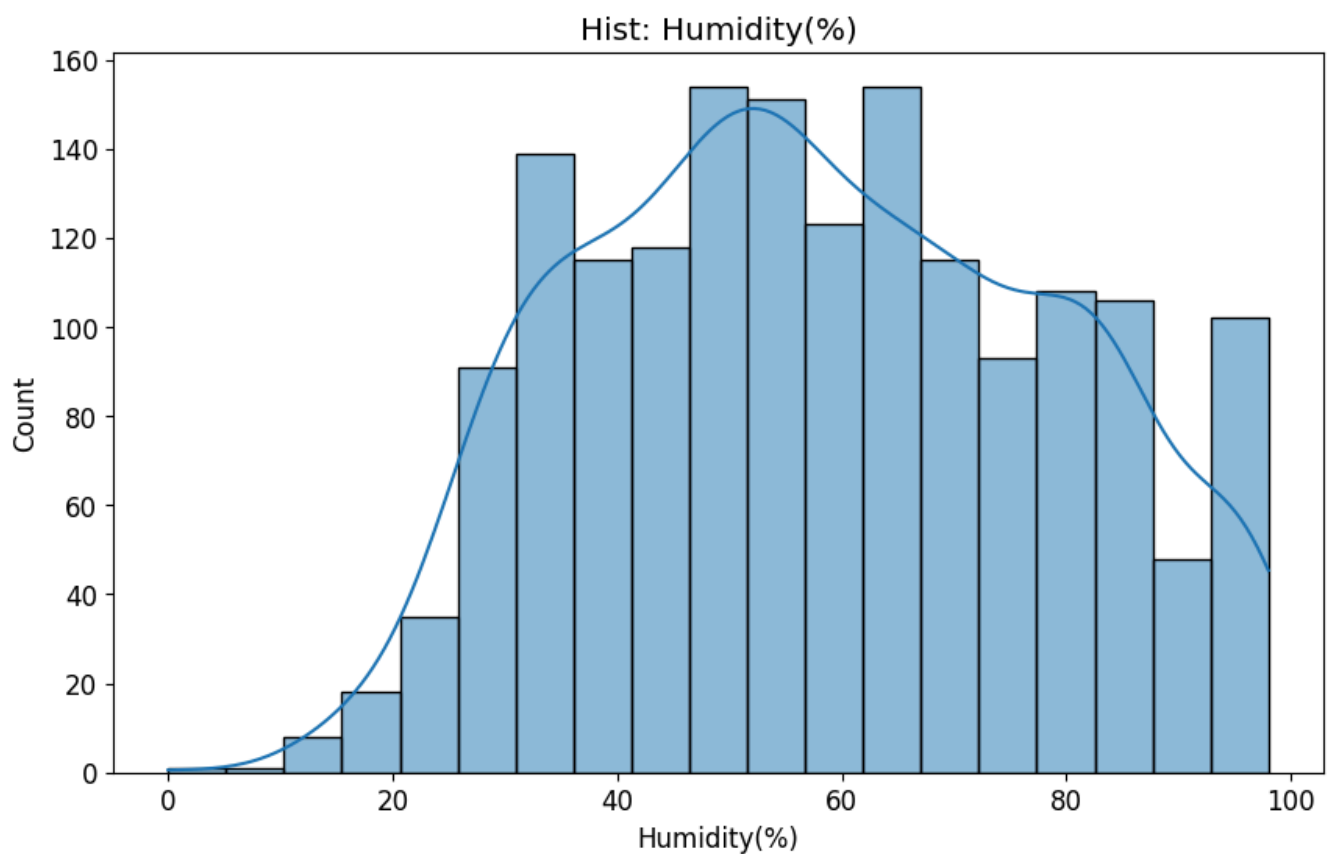
```
Temperature
count      1752.000000
mean       13.162957
std        12.024380
min        -17.800000
25%         3.900000
50%        14.450000
75%        22.900000
max        37.900000
Name: Temperature, dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -24.6 51.4
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
```

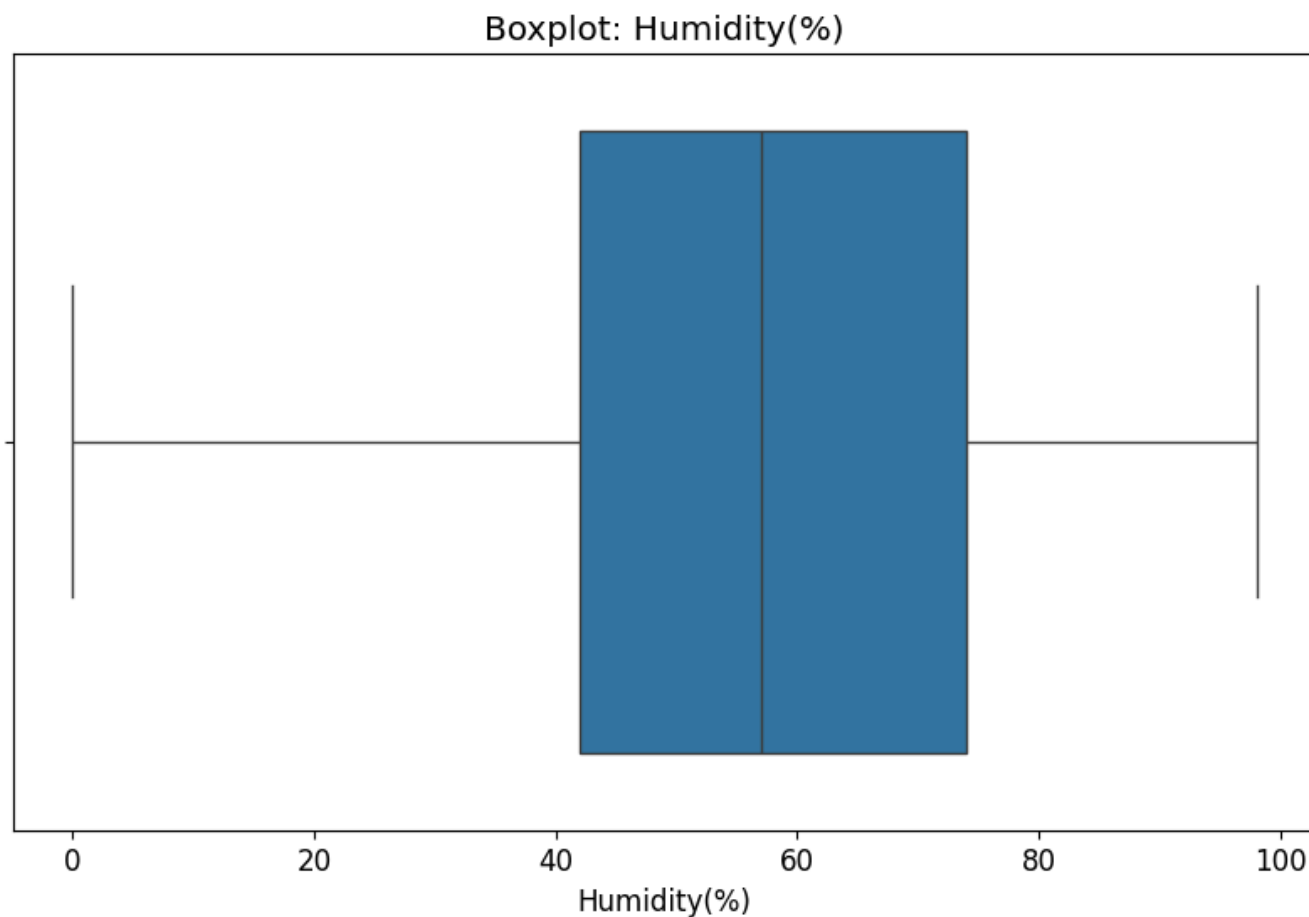


Humidity(%)

count	1680.000000
mean	58.172619
std	20.602653

```
min          0.000000
25%          42.000000
50%          57.000000
75%          74.000000
max          98.000000
Name: Humidity(%), dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -6.0 122.0
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
```





Wind speed (m/s)

count 1700.000000

mean 1.737000

std 1.013244

min 0.000000

25% 1.000000

50% 1.600000

75% 2.300000

max 5.600000

Name: Wind speed (m/s), dtype: float64

Выбросы:

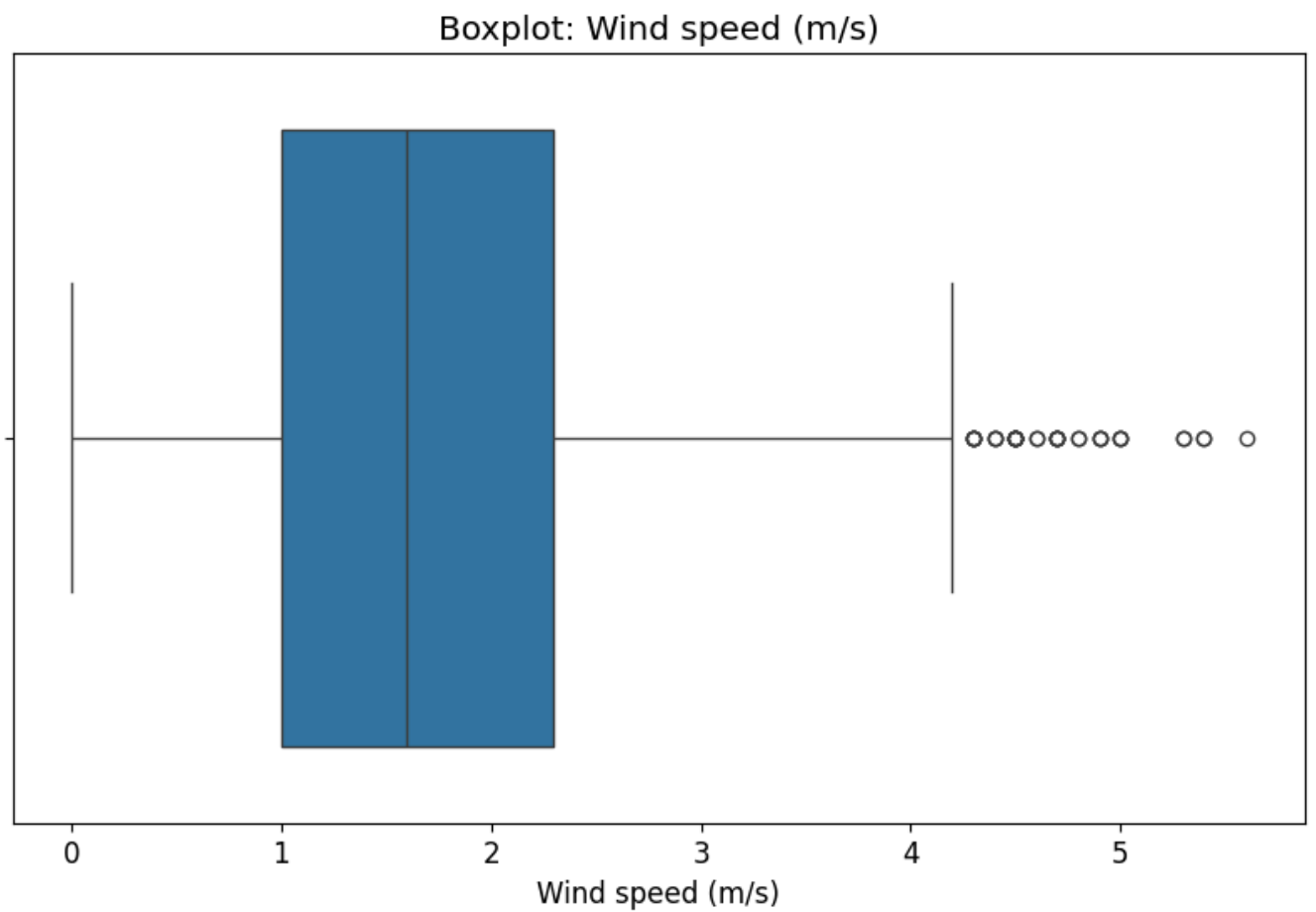
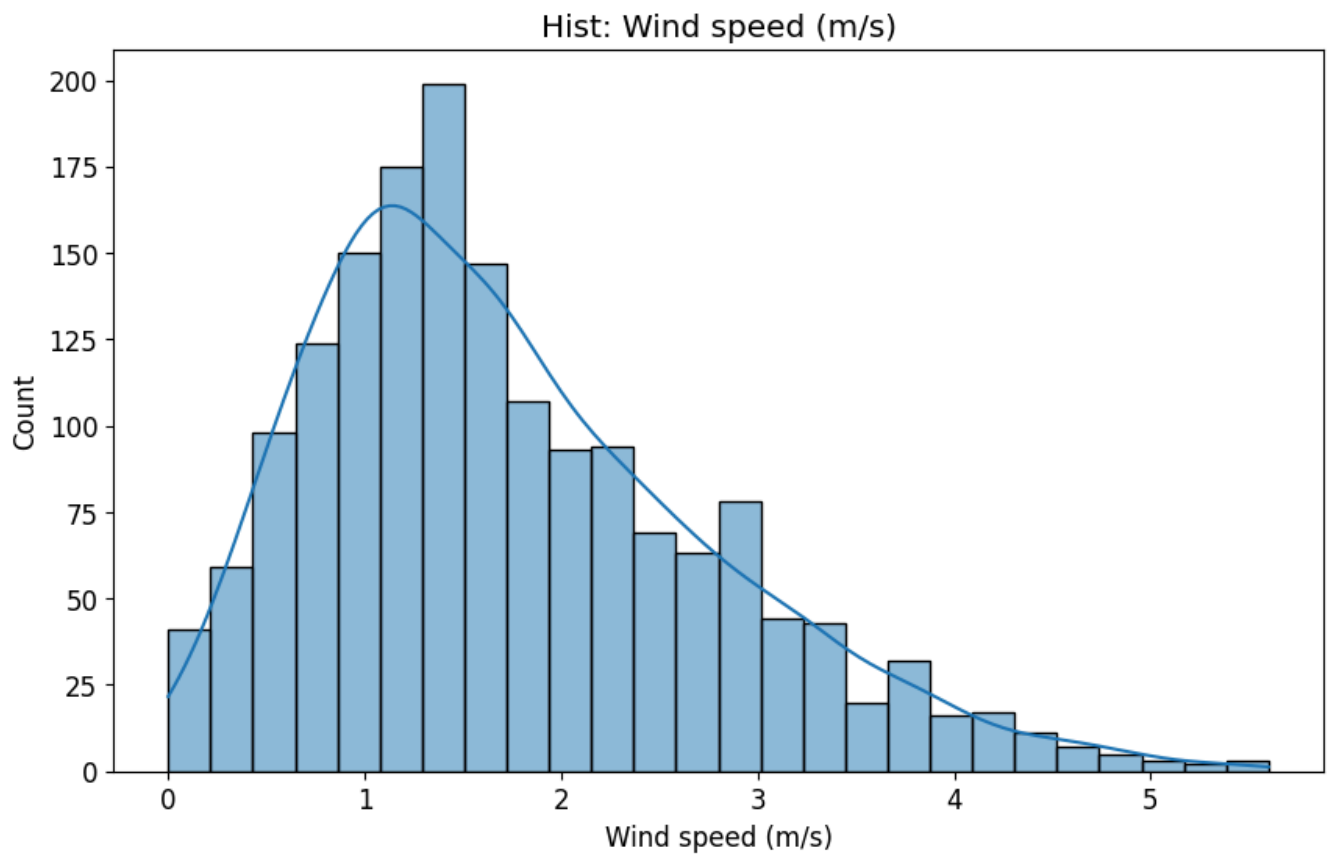
Количество: 36

Процент: 2.12

Границы: -0.95 4.25

Первые 5 выбросов выше верхней границы: [4.3 4.3 4.3 4.3 4.3]

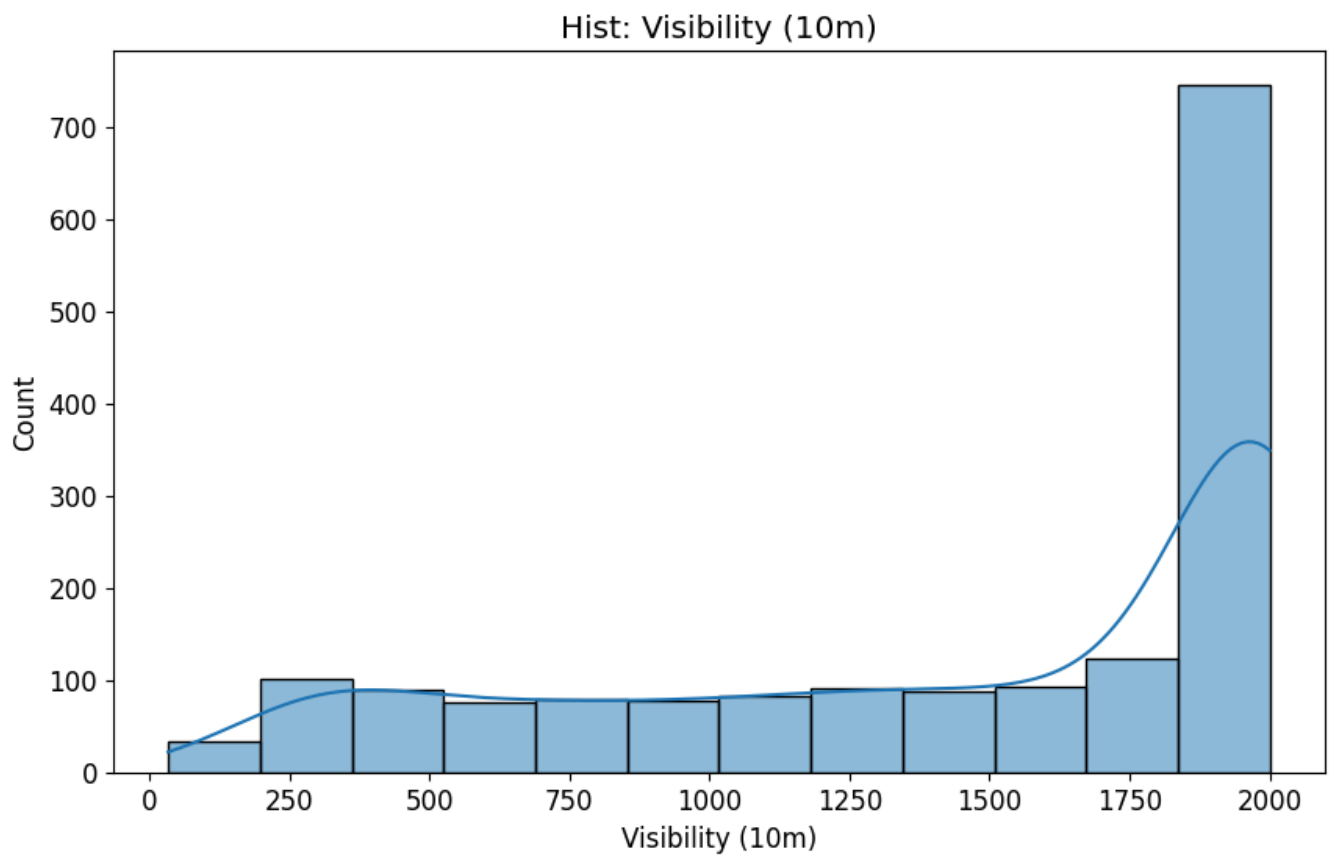
5 максимальных значений: [5.3 5.3 5.4 5.4 5.6]



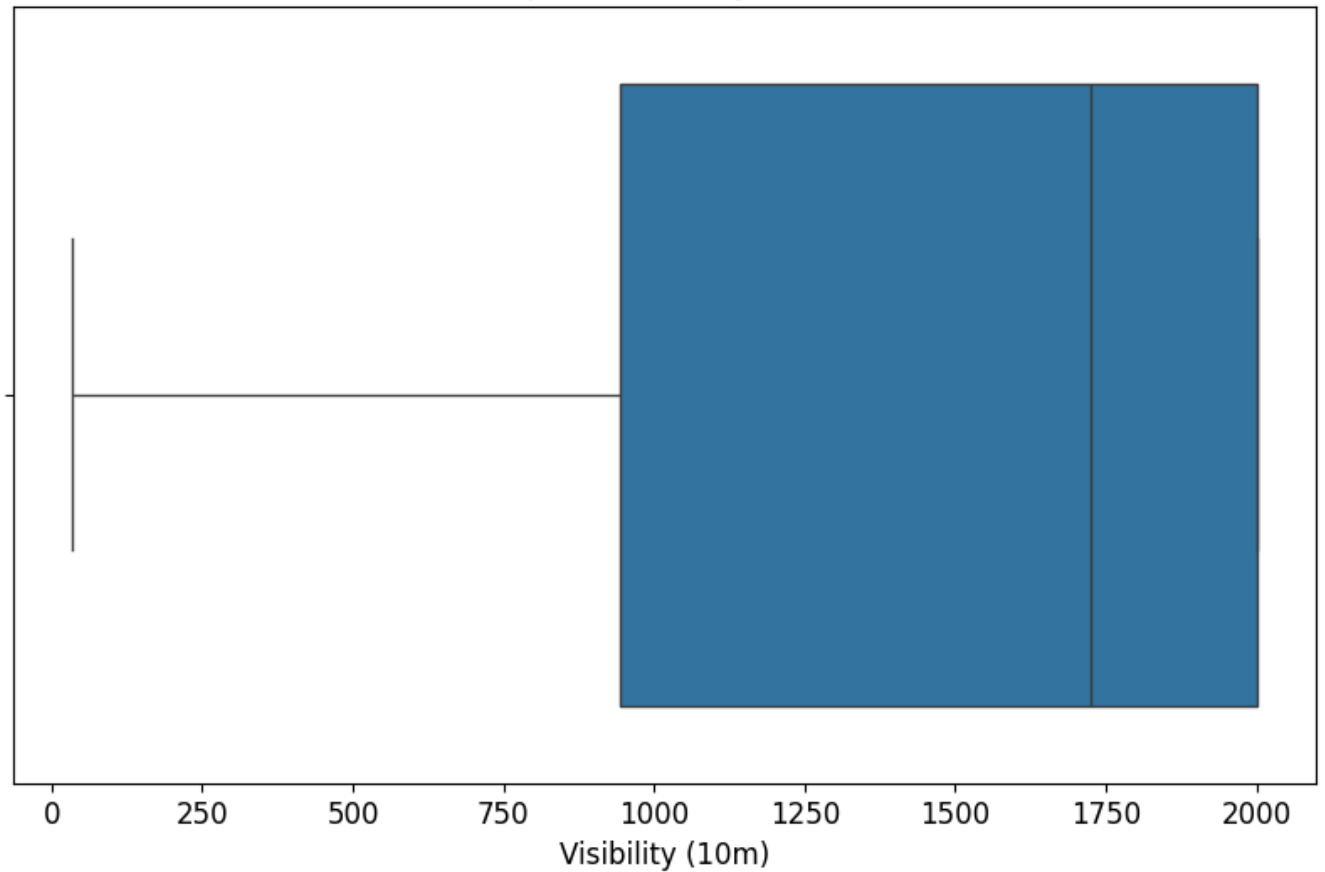
Visibility (10m)

count	1680.000000
mean	1442.061905
std	613.892076

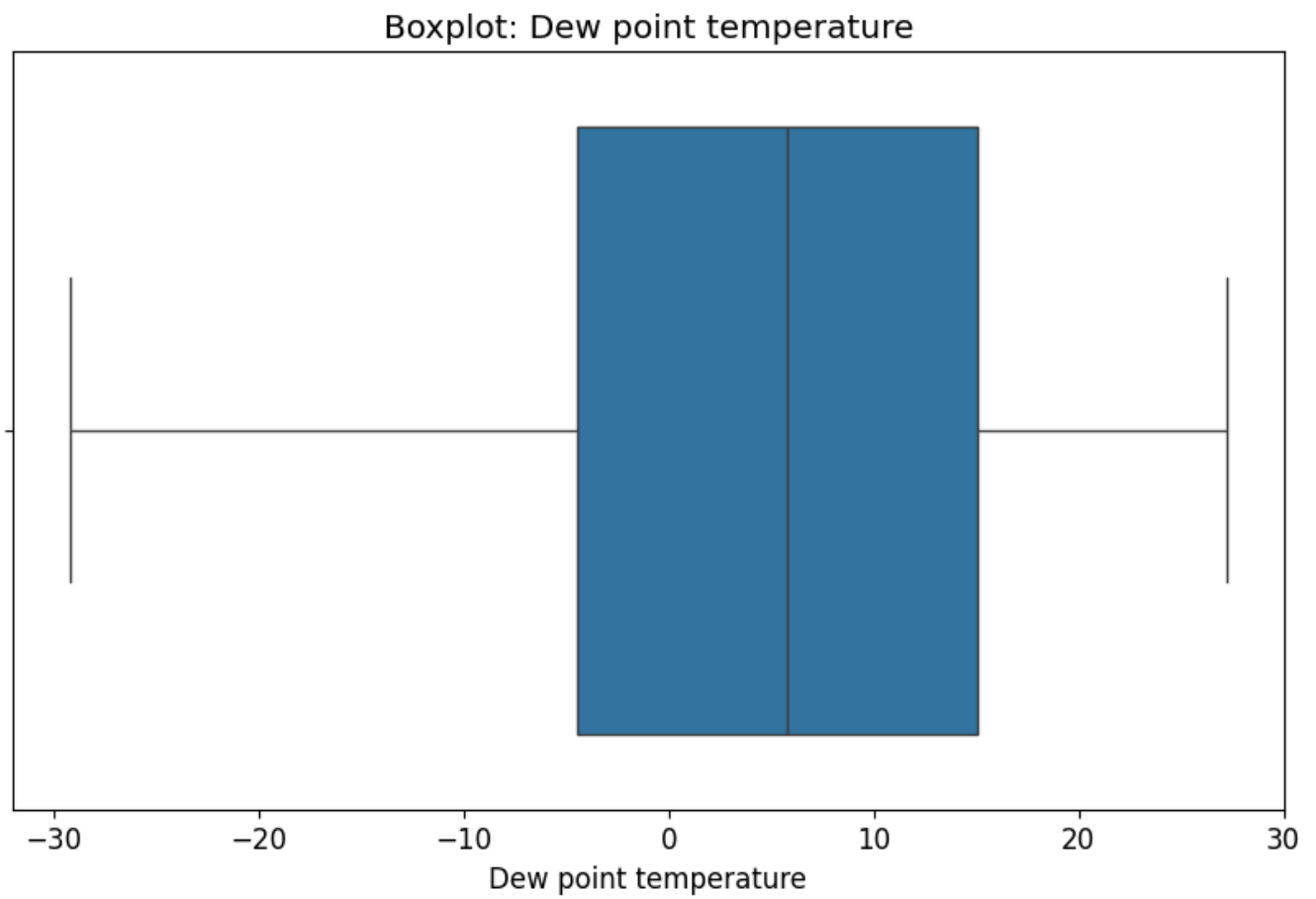
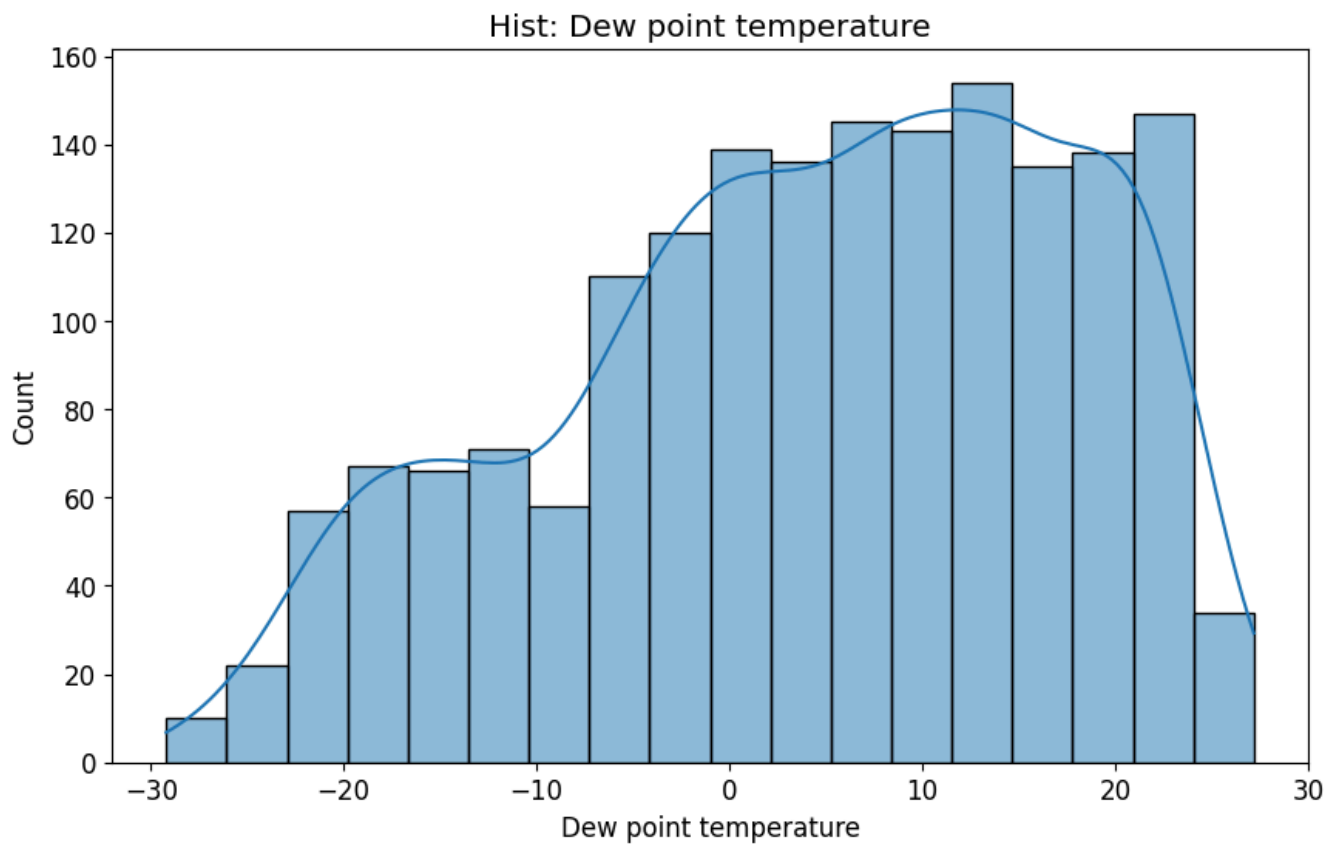
```
min          34.000000
25%          942.750000
50%         1723.500000
75%         2000.000000
max          2000.000000
Name: Visibility (10m), dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -643.12 3585.88
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
```



Boxplot: Visibility (10m)



```
Dew point temperature
count    1752.000000
mean       4.283105
std       13.169451
min       -29.200000
25%       -4.500000
50%        5.800000
75%       15.100000
max       27.200000
Name: Dew point temperature, dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -33.9 44.5
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
```



Solar Radiation (MJ/m2)

count	1700.000000
mean	0.578035
std	0.887957
min	0.000000

25% 0.000000
50% 0.020000
75% 0.910000
max 3.420000

Name: Solar Radiation (MJ/m2), dtype: float64

Выбросы:

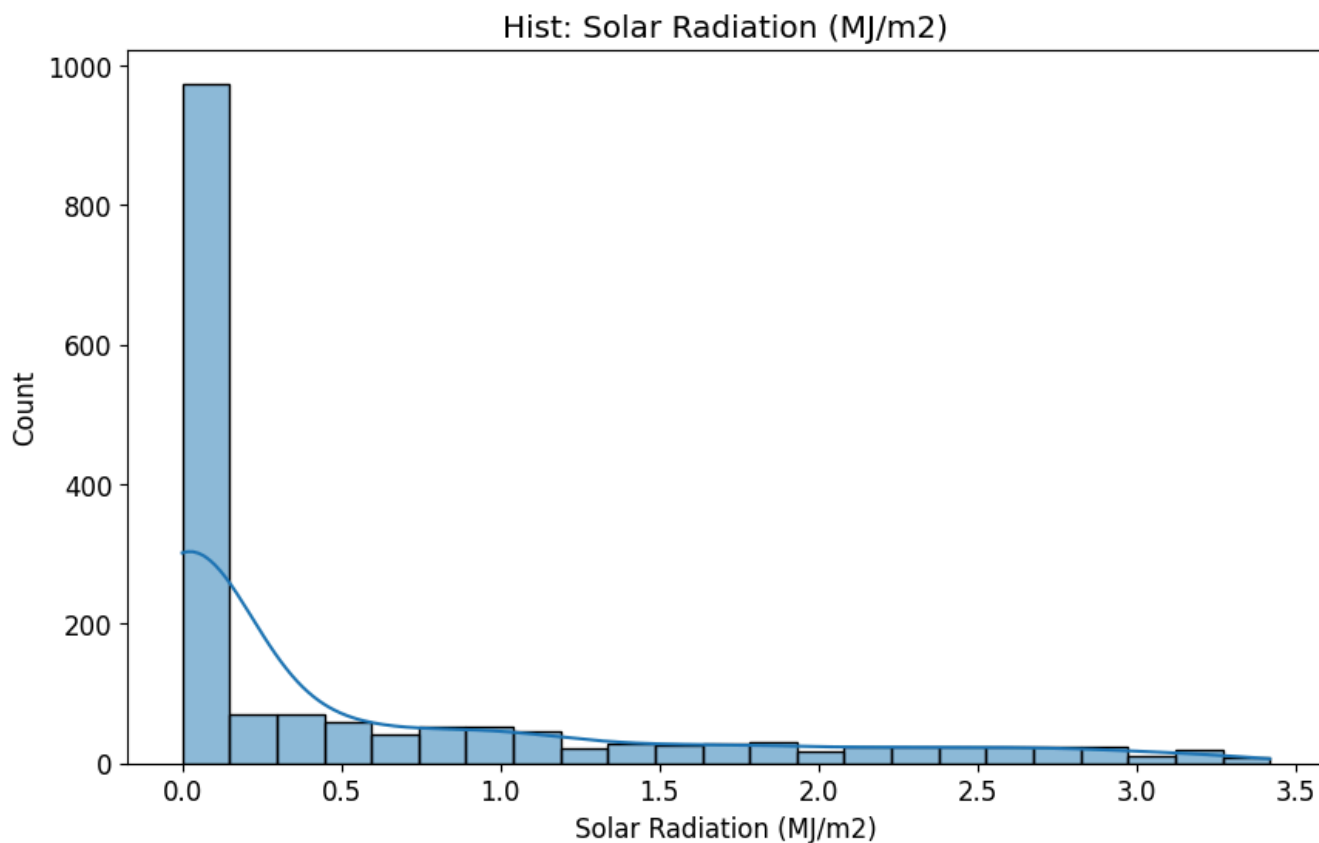
Количество: 147

Процент: 8.65

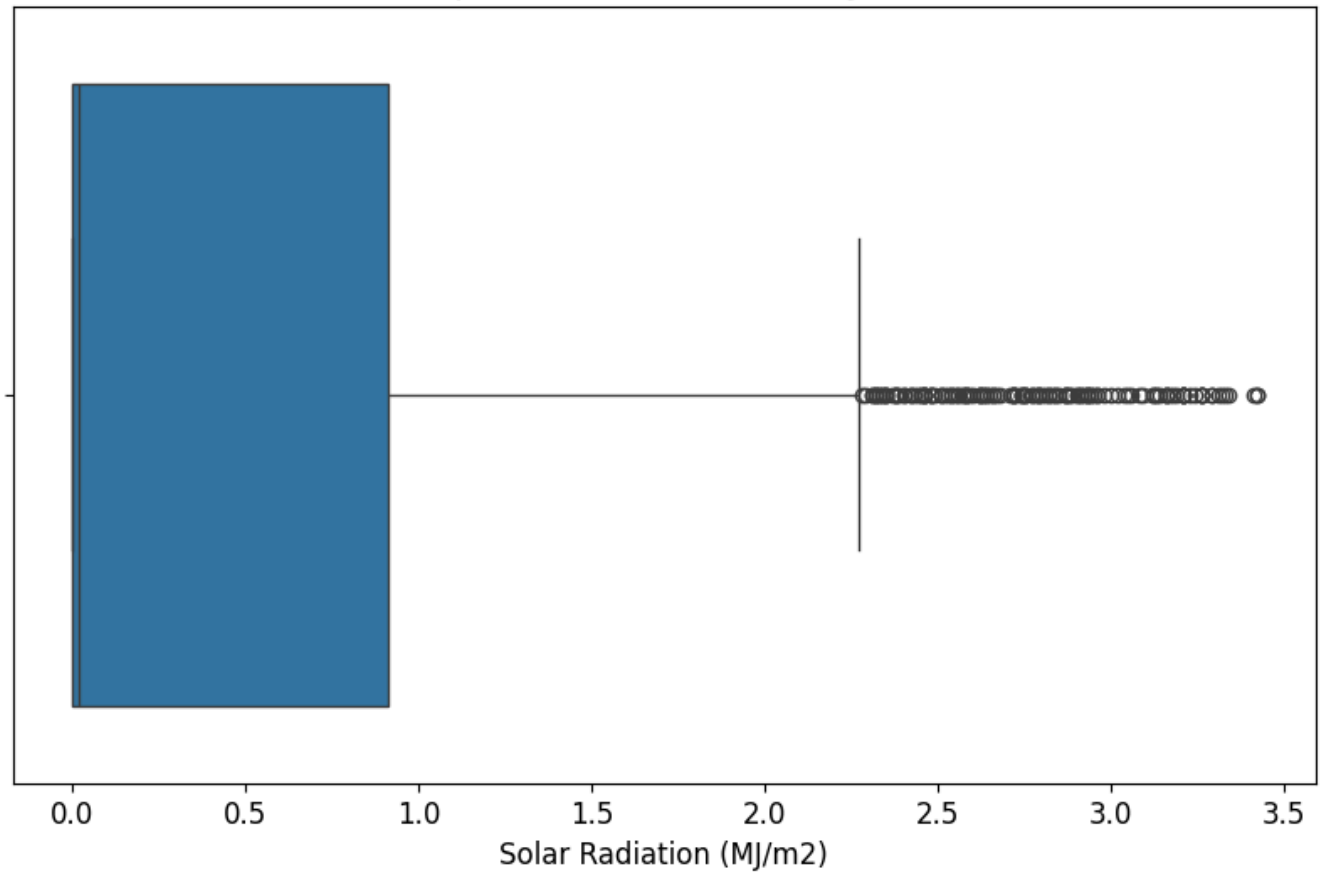
Границы: -1.36 2.28

Первые 5 выбросов выше верхней границы: [2.28 2.29 2.31 2.31 2.32]

5 максимальных значений: [3.33 3.34 3.41 3.42 3.42]



Boxplot: Solar Radiation (MJ/m2)



Rainfall(mm)

count 1688.000000

mean 0.145794

std 0.920346

min 0.000000

25% 0.000000

50% 0.000000

75% 0.000000

max 15.500000

Name: Rainfall(mm), dtype: float64

Выбросы:

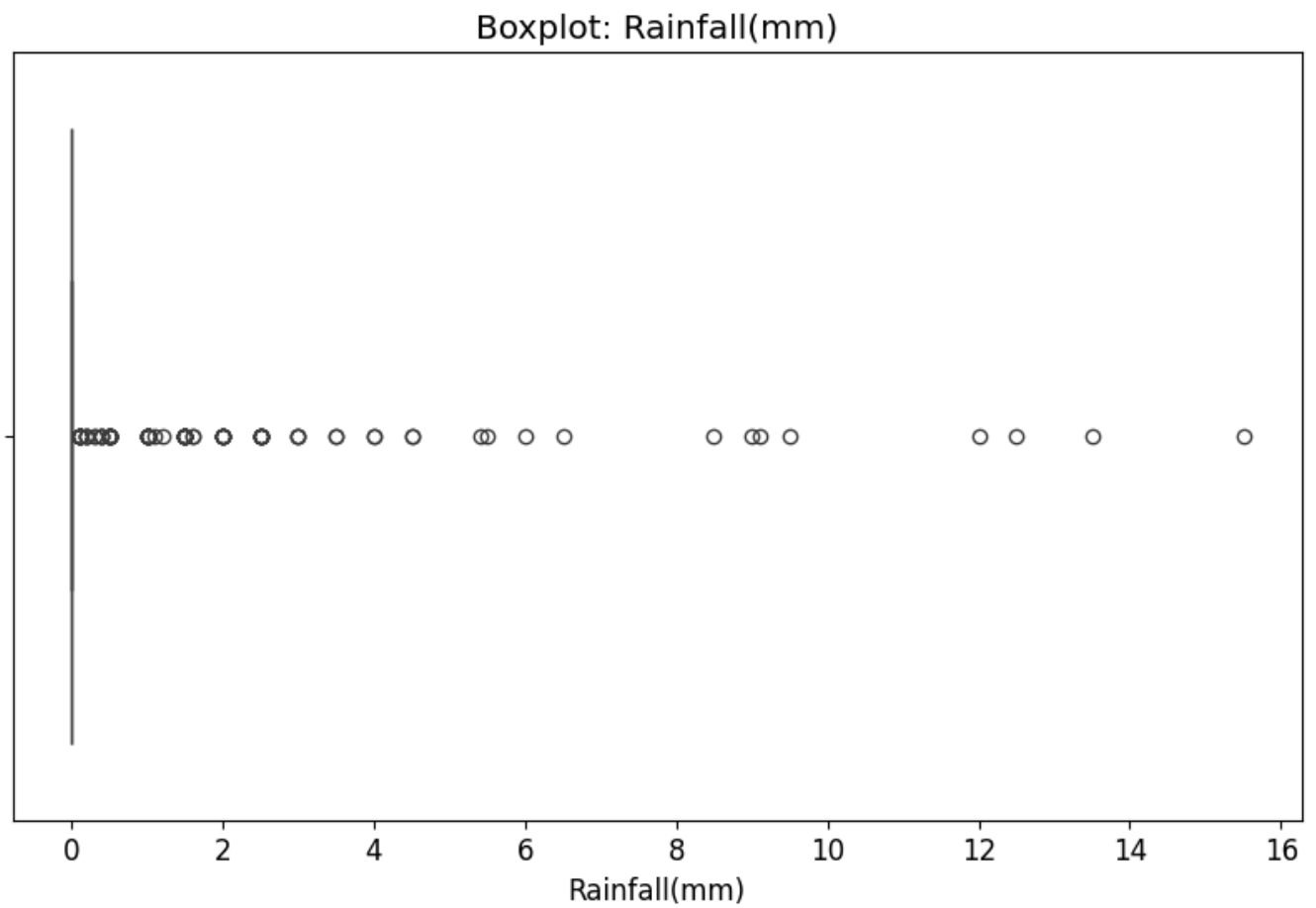
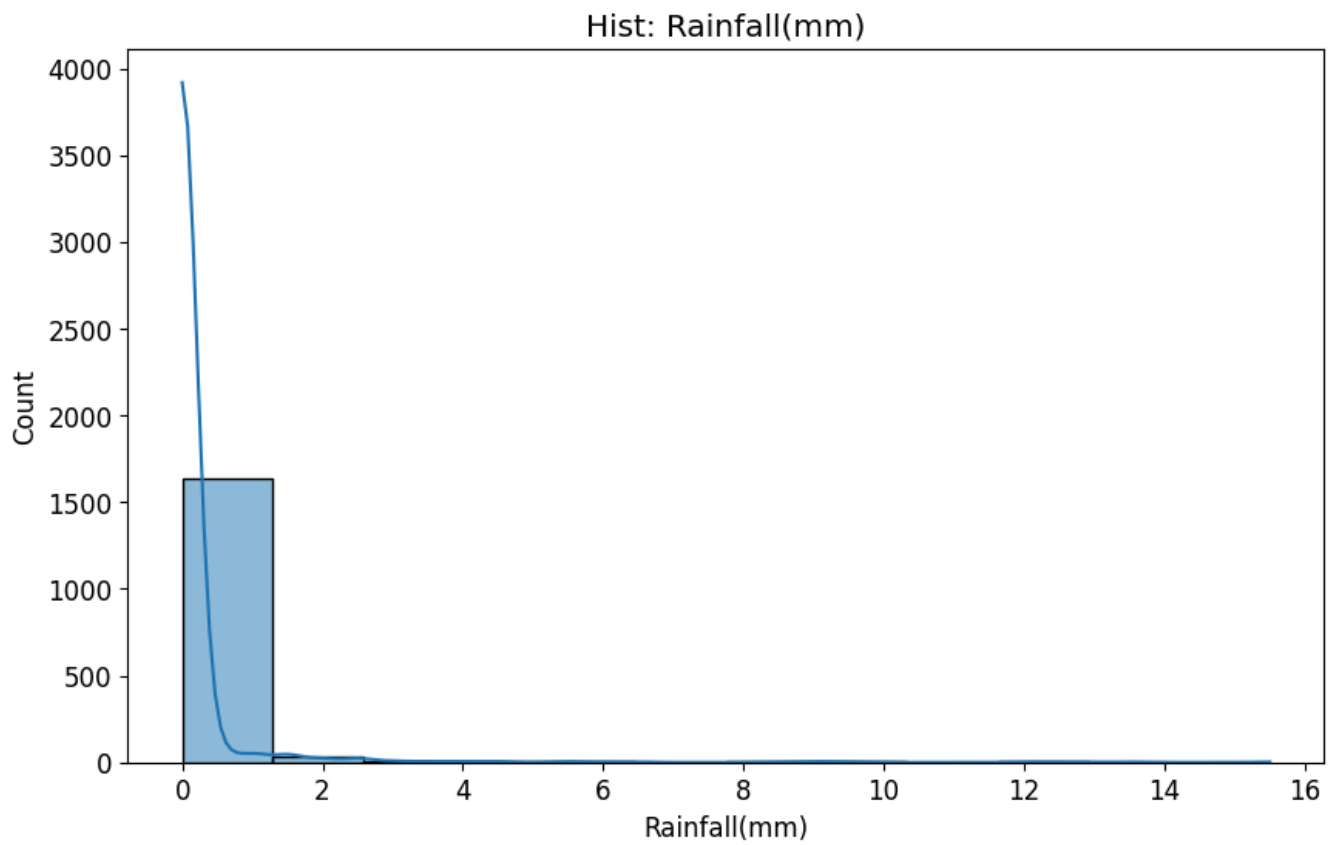
Количество: 120

Процент: 7.11

Границы: 0.0 0.0

Первые 5 выбросов выше верхней границы: [0.1 0.1 0.1 0.1 0.1]

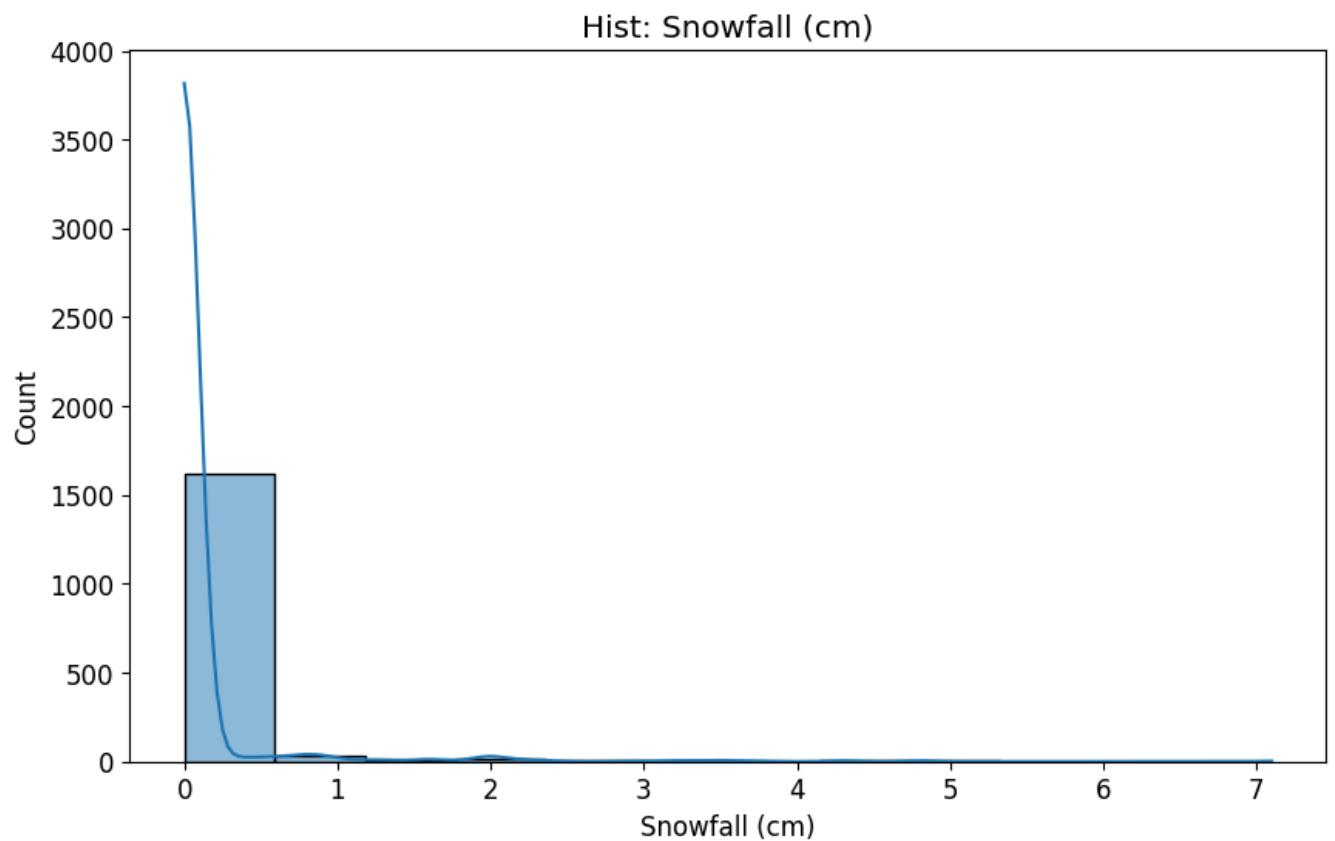
5 максимальных значений: [9.5 12. 12.5 13.5 15.5]

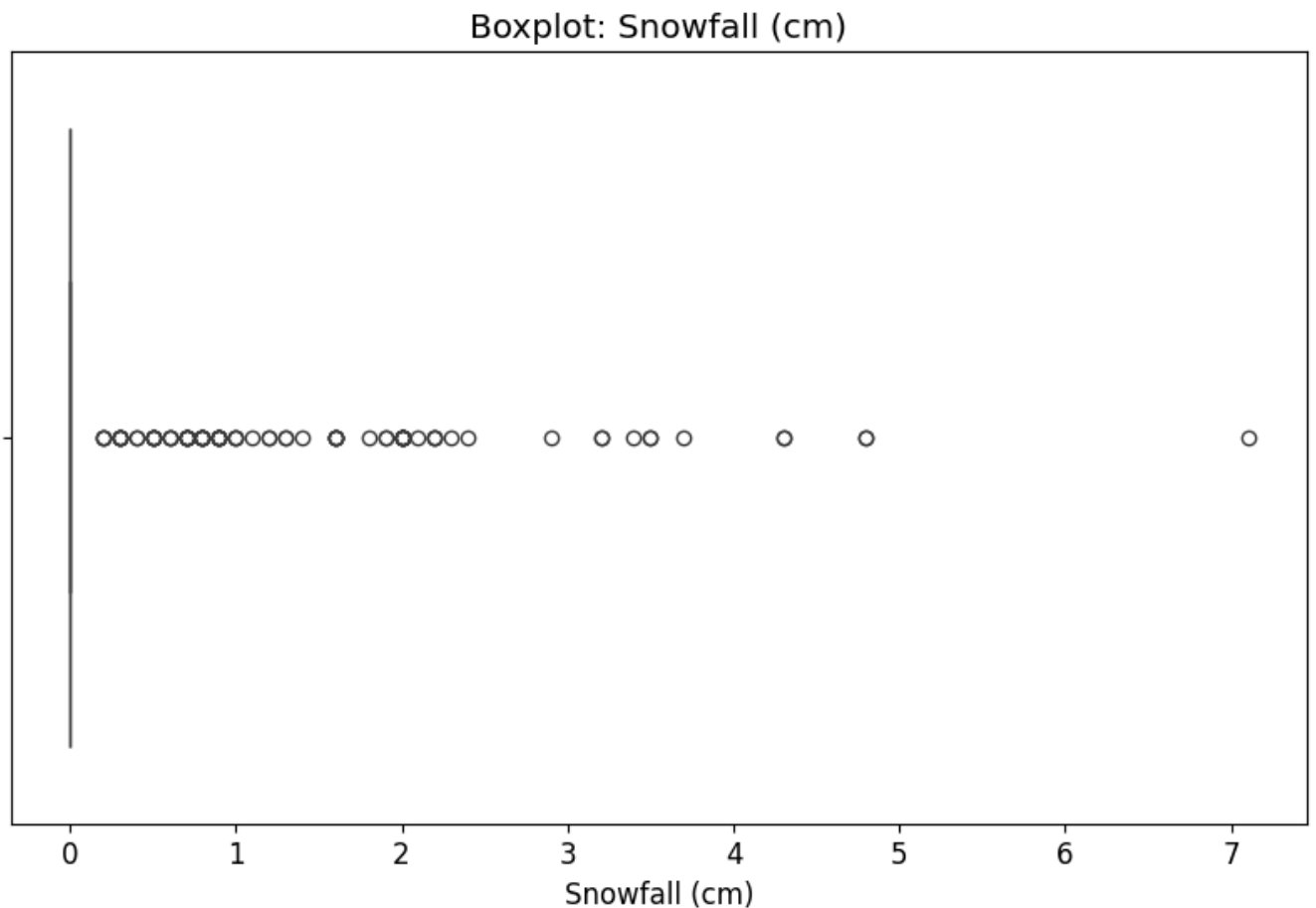


```
Snowfall (cm)
count      1691.000000
mean        0.078001
std         0.438827
```



```
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max          7.100000
Name: Snowfall (cm), dtype: float64
Выбросы:
Количество: 87
Процент: 5.14
Границы: 0.0 0.0
Первые 5 выбросов выше верхней границы: [0.2 0.2 0.2 0.3 0.3]
5 максимальных значений: [4.3 4.3 4.8 4.8 7.1]
```





Вывод по выбросам

В числовых признаках преобладает неравномерное распределение

- **Temperature, Humidity, Visibility, Dew point temperature** выбросы не выявлены по IQR
- **Wind speed** небольшое количество выбросов, обычные экстремальные значения для погоды
- **Solar Radiation** много выбросов, распределение с сильной правой асимметрией
- **Rainfall** 7% выбросов, большинство значений нулевые, есть редкие сильные осадки.
- **Snowfall** 5% выбросов, аналогично **Rainfall**

Вывод: большинство признаков устойчивы, но **Solar Radiation, Rainfall и Snowfall** имеют выбросы и сильную асимметрию распределения. Это нормально для погодных данных, и не требуют удаления

Корреляции

```
In [13]: def phik_corr_matrix(df, interval_cols=None, drop_cols=None):  
        data = df.copy()  
  
        if drop_cols:  
            data = data.drop(columns=drop_cols, errors='ignore')
```

```

corr = data.phik_matrix(interval_cols=interval_cols)
plt.figure(figsize=(14, 10))
sns.heatmap(corr, cmap='coolwarm', annot=True)
plt.show()

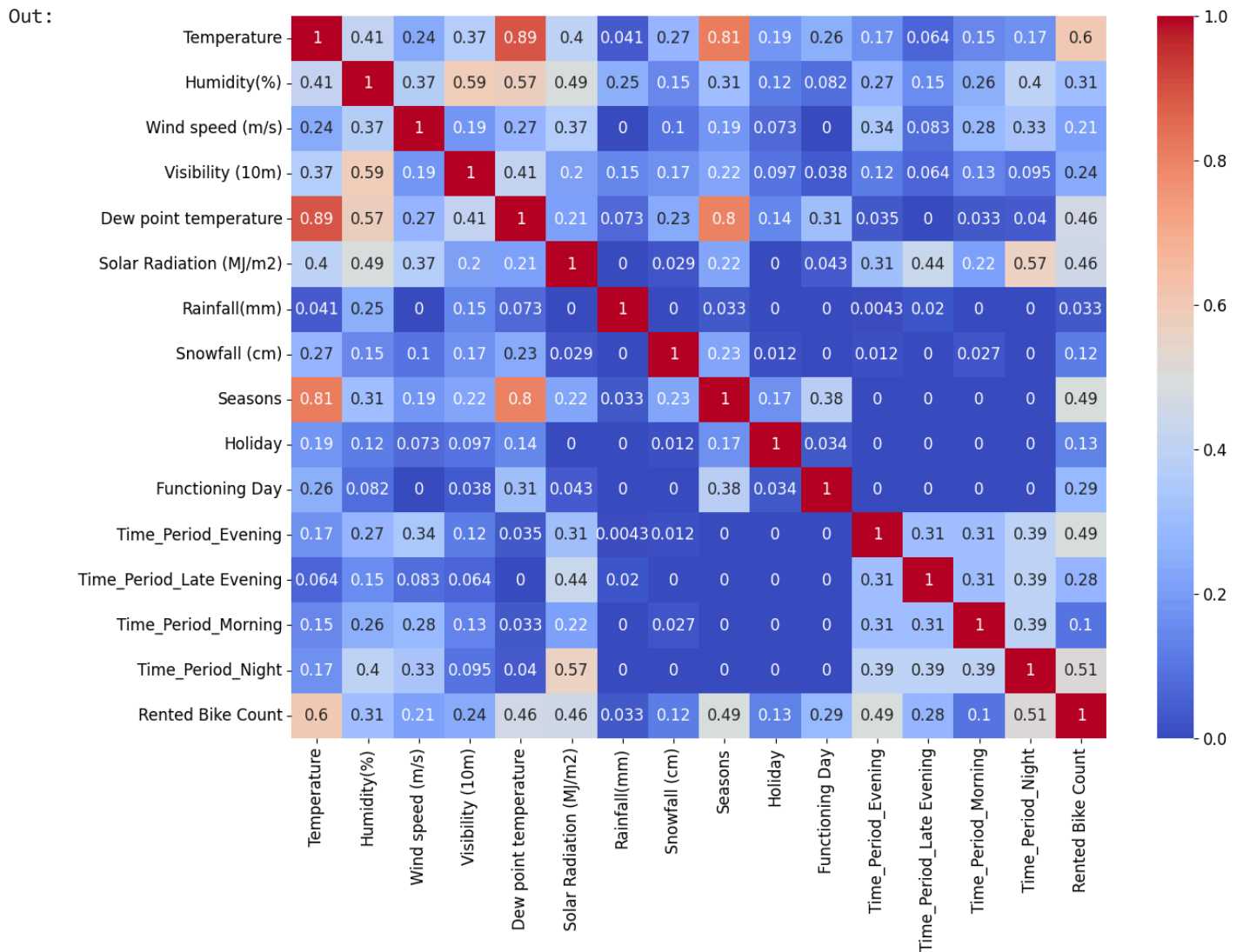
return corr

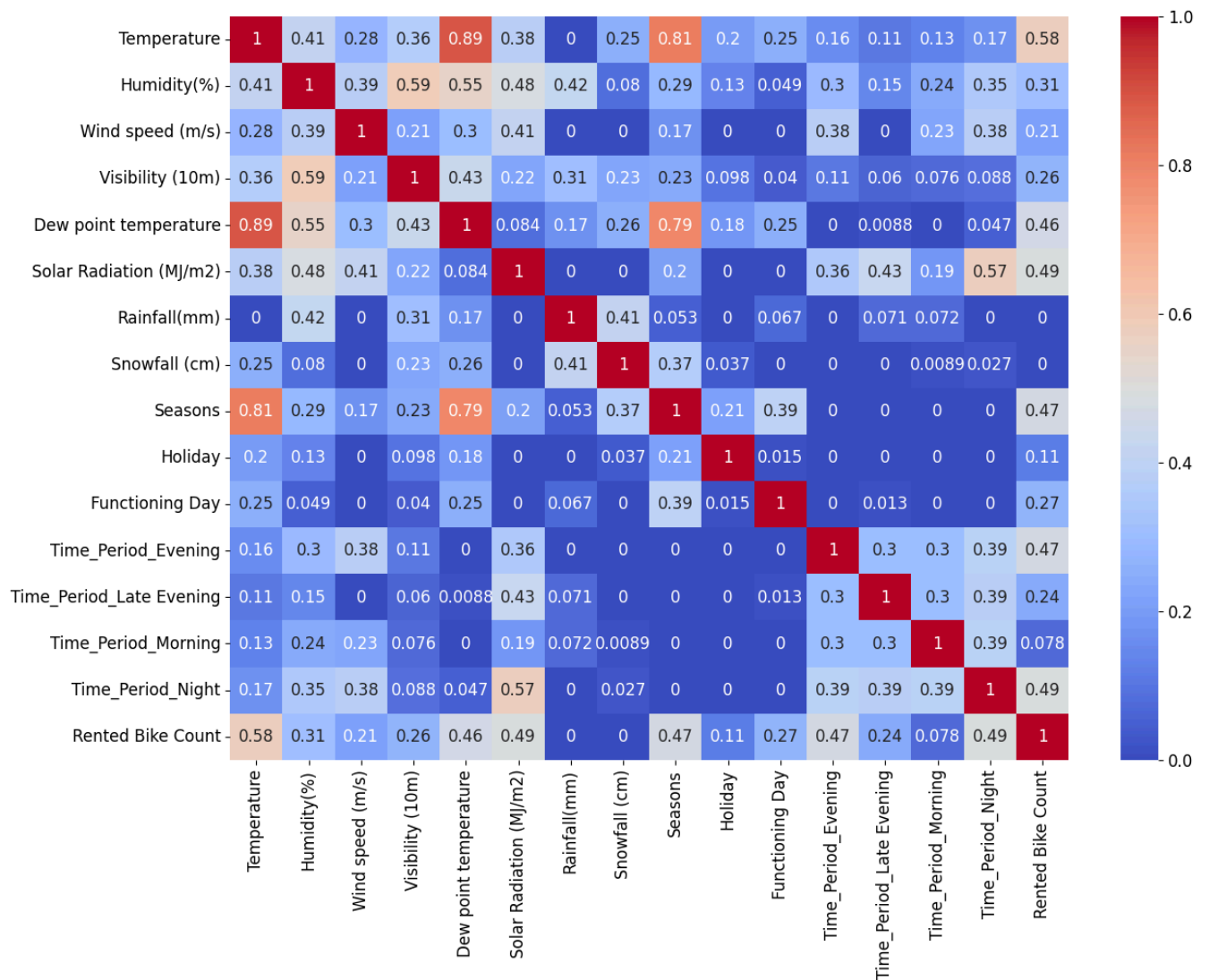
```

```

In [14]: interval_cols = ["Temperature", "Humidity(%)", "Wind speed (m/s)", "Visibility (10m)",
                           "Dew point temperature", "Solar Radiation (MJ/m2)", "Rainfall(mm)", "Snowfall (cm)", "Rented Bike Count"]
corr_train = phik_corr_matrix(train, interval_cols=interval_cols)
corr_test = phik_corr_matrix(test, interval_cols=interval_cols, drop_cols=None)

```





Построим графики для выявления зависимости спроса от различных погодных характеристик

```
In [15]: fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.ravel()

sns.lineplot(data=train, x='Temperature', y='Rented Bike Count', ax=axes[0])
axes[0].set_title('Temperature')

sns.lineplot(data=train, x='Solar Radiation (MJ/m2)', y='Rented Bike Count', ax=axes[1])
axes[1].set_title('Solar Radiation')

sns.lineplot(data=train, x='Humidity(%)', y='Rented Bike Count', ax=axes[2])
axes[2].set_title('Humidity')

sns.barplot(data=train, x='Seasons', y='Rented Bike Count', ax=axes[3])
axes[3].set_title('Seasons')
```

```

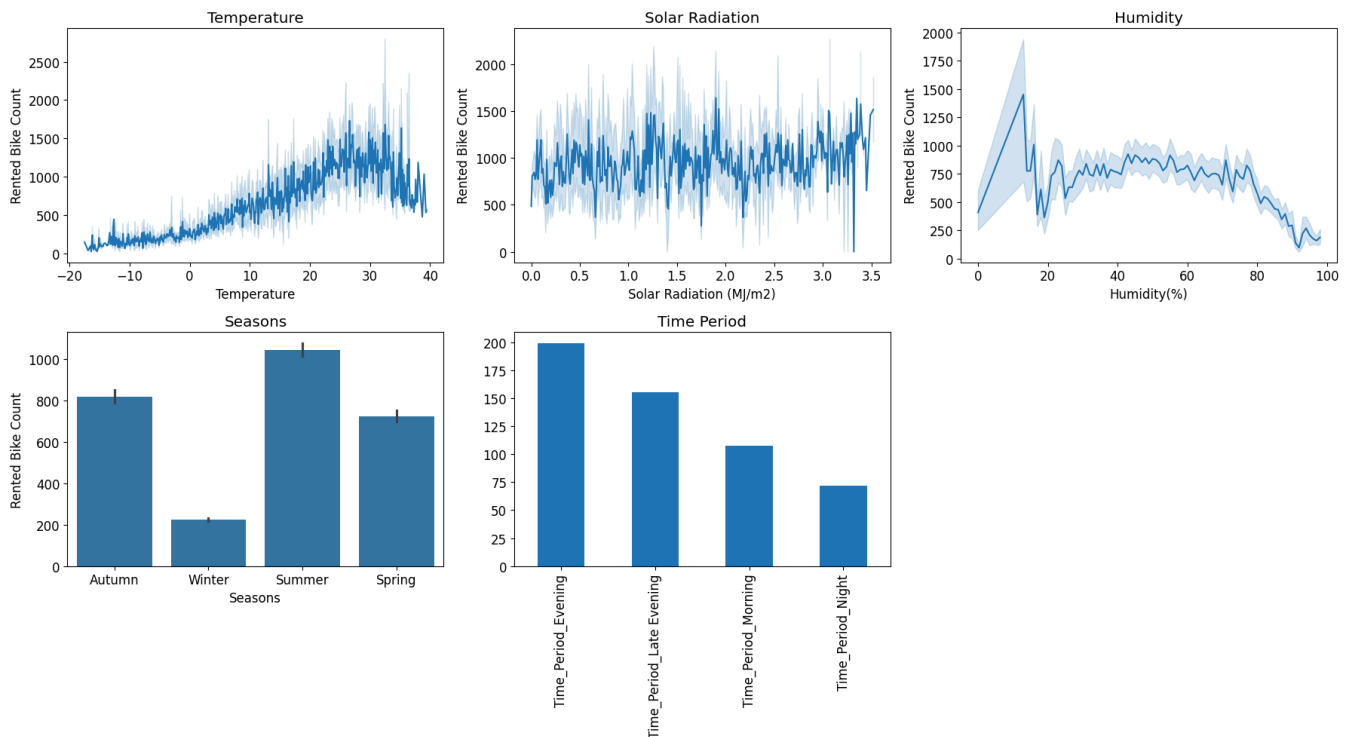
time_cols = [c for c in train.columns if 'Time_Period' in c]
train[time_cols].mul(train['Rented Bike Count'], axis=0).mean().plot(kind='bar',
ax=axes[4])
axes[4].set_title('Time Period')

fig.delaxes(axes[5])

plt.tight_layout()
plt.show()

```

Out :



Temperature и Dew point temperature сильно коллинеарны (phik 0.89), так как Dew point temperature вычисляется из температуры и влажности оставляем Temperature, Dew point temperature удаляем

```

In [16]: collinear_cols = ['Dew point temperature']

train = train.drop(columns=collinear_cols)
test = test.drop(columns=collinear_cols)

```

Вывод

Наибольшее влияние на спрос оказывает температура воздуха. Графики показывают нелинейный характер: при отрицательных температурах количество аренд минимально, потом спрос растет и достигает максимума при температуре 25–30 . При дальнейшем повышении температуры идет снижение числа поездок

Солнечная радиация так же оказывает влияние на количество аренд. При увеличении солнечной радиации спрос возрастает, однако зависимость слабее, чем для температуры, поскольку разброс значений большой

Влажность воздуха отрицательно влияет на целевую переменную. Наиболее благоприятными являются значения 40–70 %. При высокой влажности спрос снижается. Возможно, влажность выступает индикатором дождливой и пасмурной погоды, при которой использование велосипедов падает

Сезонность также влияет на спрос. Максимальное количество аренд приходится на лето, меньше на осень и весну, зимой минимальная активность

Различия спроса в зависимости от времени суток: наибольший спрос в вечерние часы, ниже поздним вечером. Утром количество ещё меньше, а ночью спрос достигает минимальных значений.

Зависимость целевой переменной от факторов окружающей среды имеет нелинейный характер. Это объясняет, почему линейные модели могут недостаточно точно прогнозировать количество аренд велосипедов. Также, влияние осадков отражается через влажность воздуха: в дождливую погоду влажность резко возрастает, поэтому именно она становится более информативным признаком для модели, чем сам факт наличия осадков

Этап 3 Предобработка: Собираем пайплайн заполняем пропуски отделяем целевую переменную

Выделим целевую переменную

```
In [17]: X_train = train.drop(columns='Rented Bike Count')
         y_train = train['Rented Bike Count']

         X_test = test.drop(columns='Rented Bike Count')
         y_test = test['Rented Bike Count']
```

Заполнение пропусков

Заполним пропуски до трансформера, так как для числовых признаков лучше считать медиану для заполнения по сезону

```
In [18]: def fill_by_group_median(df, group_col, num_cols, medians=None):
         df = df.copy()
         if medians is None:
```

```

medians = df.groupby(group_col)[num_cols].median()

global_medians = medians.median()

for col in num_cols:
    fill_values = df[group_col].map(medians[col])
    fill_values = fill_values.fillna(global_medians[col])
    df[col] = df[col].fillna(fill_values)

return df, medians

num_cols = ['Temperature', 'Humidity(%)', 'Wind speed (m/s)', 'Visibility (10m)',
            'Solar Radiation (MJ/m2)', 'Rainfall(mm)', 'Snowfall (cm)']

X_train, season_medians = fill_by_group_median(X_train, group_col='Seasons', num_cols=num_cols)

X_test, skip = fill_by_group_median(X_test, group_col='Seasons', num_cols=num_cols, medians=season_medians)

```

```

In [19]: missing_report(X_train)
print('='*50)
missing_report(X_test)

```

Out:

```

Temperature
    пропусков: 0
    доля: 0.0%
Humidity(%)
    пропусков: 0
    доля: 0.0%
Wind speed (m/s)
    пропусков: 0
    доля: 0.0%
Visibility (10m)
    пропусков: 0
    доля: 0.0%
Solar Radiation (MJ/m2)
    пропусков: 0
    доля: 0.0%
Rainfall(mm)
    пропусков: 0
    доля: 0.0%
Snowfall (cm)
    пропусков: 0

```

```
        доля: 0.0%
Seasons
        пропусков: 0
        доля: 0.0%
Holiday
        пропусков: 0
        доля: 0.0%
Functioning Day
        пропусков: 0
        доля: 0.0%
Time_Period_Evening
        пропусков: 0
        доля: 0.0%
Time_Period_Late Evening
        пропусков: 0
        доля: 0.0%
Time_Period_Morning
        пропусков: 0
        доля: 0.0%
Time_Period_Night
        пропусков: 0
        доля: 0.0%
Без удаления пропусков строк: 7008
После удаления: 7008
Потеря: 0
Доля потерь: 0.0
=====
Temperature
        пропусков: 0
        доля: 0.0%
Humidity(%)
        пропусков: 0
        доля: 0.0%
Wind speed (m/s)
        пропусков: 0
        доля: 0.0%
Visibility (10m)
        пропусков: 0
        доля: 0.0%
Solar Radiation (MJ/m2)
        пропусков: 0
        доля: 0.0%
Rainfall(mm)
        пропусков: 0
        доля: 0.0%
Snowfall (cm)
        пропусков: 0
        доля: 0.0%
Seasons
        пропусков: 0
```



```
        доля: 0.0%
Holiday
        пропусков: 0
        доля: 0.0%
Functioning Day
        пропусков: 0
        доля: 0.0%
Time_Period_Evening
        пропусков: 0
        доля: 0.0%
Time_Period_Late Evening
        пропусков: 0
        доля: 0.0%
Time_Period_Morning
        пропусков: 0
        доля: 0.0%
Time_Period_Night
        пропусков: 0
        доля: 0.0%
Без удаления пропусков строк: 1752
После удаления: 1752
Потеря: 0
Доля потерь: 0.0
```

Создание пайплайна для обработки категориальных признаков и заполнения пропусков в категориальных признаках

```
In [20]: def prepare_pipeline(num_cols, cat_cols, ordinal_cols=None, target_cols=None, missing_num="median",
                                scaling="standard", missing_cat='most_frequent', encode="one
hot", collinear_cols=None):
    if collinear_cols is None:
        collinear_cols = []
    if ordinal_cols is None:
        ordinal_cols = []
    if target_cols is None:
        target_cols = []

    num_cols = [c for c in num_cols if c not in collinear_cols]
    cat_cols = [c for c in cat_cols if c not in collinear_cols]
    target_cols = [c for c in target_cols if c not in collinear_cols]

    num_steps = [("imputer", SimpleImputer(strategy=missing_num))]
    if scaling == "standard":
        num_steps.append(("scaler", StandardScaler()))
```

```

elif scaling == "minmax":
    num_steps.append(("scaler", MinMaxScaler()))
num_pipe = Pipeline(num_steps)

cat_steps = [("imputer", SimpleImputer(strategy=missing_cat))]
if encode == "onehot":
    cat_steps.append(("encoder", OneHotEncoder(handle_unknown="ignore", s
parse_output=False)))
cat_pipe = Pipeline(cat_steps)

transformers = [("num", num_pipe, num_cols),
                ("cat", cat_pipe, cat_cols),]

for col, categories in ordinal_cols:
    ordinal_pipe = Pipeline([ ("imputer", SimpleImputer(strategy="most_frequen
t")),
                              ("encoder", OrdinalEncoder(categories=[categori
es]))])

    transformers.append(("ord_"+col, ordinal_pipe, [col]))

if target_cols:
    target_pipe = Pipeline([("encoder", TargetEncoder(handle_unknown="value",
handle_missing="value"))])
    transformers.append(("target", target_pipe, target_cols))

preprocessor = ColumnTransformer(transformers)
return preprocessor

```

```
In [21]: cat_cols = ['Seasons', 'Holiday', 'Functioning Day']
```

```
pass_cols = [col for col in X_train.columns if 'Time_Period_' in col]
```

```
preprocessor = prepare_pipeline(num_cols=num_cols, cat_cols=cat_cols, missing_num
='median', scaling='standard', missing_cat='most_frequent', encode='onehot')
```

```
transformers = preprocessor.transformers
```

```
preprocessor = ColumnTransformer(transformers=[*transformers, ('pass', 'passthrou
gh', pass_cols)])
```

```
X_train_prepared = preprocessor.fit_transform(X_train, y_train)
```

```
X_test_prepared = preprocessor.transform(X_test)
```

```
print("Размер после предобработки:")
```

```
print("Train:", X_train_prepared.shape)
print("Test:", X_test_prepared.shape)
```

Out:

```
Размер после предобработки:
Train: (7008, 19)
Test: (1752, 19)
```

Обучение базовых моделей: построение пайплайнов для KNN и Decision Tree

```
In [22]: def evaluate_models(models, X, y, preprocessor, selector=None):
cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
results = []
for name, model in models.items():
    steps = []
    if preprocessor is not None:
        steps.append(("preprocessor", preprocessor))
    if selector is not None:
        steps.append(("selector", selector))
    steps.append(("model", model))
    pipe = Pipeline(steps)
    y_pred = cross_val_predict(pipe, X, y, cv=cv, n_jobs=-1)
    mae = mean_absolute_error(y, y_pred)
    mse = mean_squared_error(y, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y, y_pred)
    result = {"model": name, "MAE": mae, "MSE": mse, "RMSE": rmse, "R2": r2}
    results.append(result)

    return pd.DataFrame(results)
```

```
In [23]: models = {
    "Linear Regression": LinearRegression(),
    "KNN": KNeighborsRegressor(n_neighbors=5),
    "Decision Tree": DecisionTreeRegressor(max_depth=5, random_state=RANDOM_STATE)
}

results_baseline = evaluate_models(models=models, X=X_train, y=y_train, preproces
```

```
sor=preprocessor)
results_baseline
```

Out:

	model	MAE	MSE	RMSE	R2
0	Linear Regression	309.983560	171261.632581	413.837689	0.589949
1	KNN	219.885046	108426.400445	329.281643	0.740395
2	Decision Tree	263.219308	144967.558183	380.746055	0.652905

Вывод по базовым моделям

Линейная регрессия показывает худший результат ($R^2 = 0.59$) модель требует линейную связь между признаками и спросом, а EDA показал нелинейные зависимости. Линейная регрессия не улавливает такие зависимости спроса

KNN с базовыми параметрами показывает лучший результат среди трёх моделей ($R^2 = 0.74$, MAE 218) метод ближайших соседей хорошо улавливает нелинейные закономерности

Decision Tree с глубиной `max_depth=5` превосходит линейную регрессию ($R^2 = 0.65$ против 0.59). Дерево способно учитывать нелинейные связи признаков, но пока работает менее точно, чем KNN. Модели требуется настройка гиперпараметров

Этап 5 Новые признаки: генерируем дополнительные признаки на основе уже имеющихся, проверяем помогают ли они модели.

1. `temp_feeling` ощущаемая температура с учётом ветрового охлаждения (ветер усиливает ощущение холода при низкой `t` и смягчает жару)
2. `temp_deviation` отклонения температуры от комфортной зоны (25°C)
3. `solar_humidity_fix` высокая влажность гасит эффект солнца

```
In [24]: def add_features(df):
          df = df.copy()
          df['temp_feeling'] = df['Temperature'] - df['Wind speed (m/s)'] * 1.5
          df['temp_deviation'] = (df['Temperature'] - 25) ** 2
          df['solar_humidity_fix'] = df['Solar Radiation (MJ/m2)'] * (1 - df['Humidity (%)'] / 100)
          return df
```

```

In [25]: X_train = add_features(X_train)
         X_test = add_features(X_test)

         num_cols_extended = num_cols + ['temp_feeling', 'temp_deviation', 'solar_humidity_fix']

         preprocessor = prepare_pipeline(num_cols=num_cols_extended, cat_cols=cat_cols, missing_num='median',
                                         scaling='standard', missing_cat='most_frequent',
                                         encode='onehot')
         transformers = preprocessor.transformers
         preprocessor = ColumnTransformer(transformers=[*transformers, ('pass', 'passthrough', pass_cols)])

         X_train_prepared = preprocessor.fit_transform(X_train, y_train)
         X_test_prepared = preprocessor.transform(X_test)

         print("Train:", X_train_prepared.shape)
         print("Test:", X_test_prepared.shape)

```

Out:

```

Train: (7008, 22)
Test: (1752, 22)

```

```

In [26]: results_baseline = evaluate_models(models=models, X=X_train, y=y_train, preprocessor=preprocessor)
         results_baseline

```

Out:

	model	MAE	MSE	RMSE	R2
0	Linear Regression	309.204536	170078.301936	412.405507	0.592782
1	KNN	215.602854	105985.146473	325.553600	0.746240
2	Decision Tree	262.705797	144316.454229	379.890055	0.654464

Вывод

Добавление новых признаков дало небольшой прирост по метрикам, оставляем их в датасетах

Этап 6. Подбор гиперпараметров: оптимизация параметров моделей с помощью библиотеки Optuna.

```
In [27]: cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
         scorer = make_scorer(mean_absolute_error, greater_is_better=False)

def objective_tree(trial):
    params = {
        "max_depth": trial.suggest_int("max_depth", 3, 20),
        "min_samples_split": trial.suggest_int("min_samples_split", 2, 50),
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 30),
        "max_features": trial.suggest_categorical("max_features", [None, "sqrt",
"log2"])
    }
    model = DecisionTreeRegressor(random_state=RANDOM_STATE, **params)
    pipeline = Pipeline([("preprocessor", preprocessor), ("model", model)])
    scores = cross_val_score(pipeline, X_train, y_train, cv=cv, scoring=scorer)
    return -scores.mean()

sampler_tree = optuna.samplers.TPESampler(seed=RANDOM_STATE)
study_tree = optuna.create_study(direction="minimize", sampler=sampler_tree)
study_tree.optimize(objective_tree, n_trials=15, show_progress_bar=True)

print("Лучшие параметры Decision Tree:", study_tree.best_params)
print("Лучший MAE:", study_tree.best_value)
```

Out:

```
[I 2026-06-26 07:45:54,898] A new study created in memory with name: no-name-8e3b319c-ffa
c-4065-aa95-0db77b8f61a1
```

```
0%|          | 0/15 [00:00<?, ?it/s]
```

```
[I 2026-06-26 07:45:55,250] Trial 0 finished with value: 214.956993007979 and parameters:
{'max_depth': 9, 'min_samples_split': 48, 'min_samples_leaf': 22, 'max_features': None}.
Best is trial 0 with value: 214.956993007979.
```

```
[I 2026-06-26 07:45:55,471] Trial 1 finished with value: 338.35865950225116 and parameter
s: {'max_depth': 4, 'min_samples_split': 44, 'min_samples_leaf': 19, 'max_features': 'log
2'}. Best is trial 0 with value: 214.956993007979.
```

```
[I 2026-06-26 07:45:55,703] Trial 2 finished with value: 238.01400257079905 and parameter
s: {'max_depth': 17, 'min_samples_split': 12, 'min_samples_leaf': 6, 'max_features': 'log
2'}. Best is trial 0 with value: 214.956993007979.
```

```
[I 2026-06-26 07:45:55,920] Trial 3 finished with value: 257.4793787385996 and parameter
s: {'max_depth': 10, 'min_samples_split': 16, 'min_samples_leaf': 19, 'max_features': 'lo
g2'}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:56,170] Trial 4 finished with value: 231.69812910105128 and parameter
s: {'max_depth': 11, 'min_samples_split': 40, 'min_samples_leaf': 6, 'max_features': 'sq
r t'}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:56,400] Trial 5 finished with value: 235.08920679618168 and parameter
s: {'max_depth': 13, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sq
r t'}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:56,619] Trial 6 finished with value: 257.00360208905647 and parameter
s: {'max_depth': 8, 'min_samples_split': 6, 'min_samples_leaf': 21, 'max_features': 'log
2'}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:56,860] Trial 7 finished with value: 330.81620212961843 and parameter
s: {'max_depth': 3, 'min_samples_split': 46, 'min_samples_leaf': 8, 'max_features': Non
e}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:57,084] Trial 8 finished with value: 255.86077231082822 and parameter
s: {'max_depth': 12, 'min_samples_split': 11, 'min_samples_leaf': 30, 'max_features': 'sq
r t'}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:57,312] Trial 9 finished with value: 236.35195700803297 and parameter
s: {'max_depth': 13, 'min_samples_split': 47, 'min_samples_leaf': 3, 'max_features': 'log
2'}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:57,652] Trial 10 finished with value: 219.0813265001794 and parameter
s: {'max_depth': 20, 'min_samples_split': 29, 'min_samples_leaf': 28, 'max_features': Non
e}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:57,975] Trial 11 finished with value: 219.806075099297 and parameter
s: {'max_depth': 19, 'min_samples_split': 31, 'min_samples_leaf': 29, 'max_features': Non
e}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:58,279] Trial 12 finished with value: 227.92292800170563 and paramete
rs: {'max_depth': 7, 'min_samples_split': 27, 'min_samples_leaf': 25, 'max_features': Non
e}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:58,625] Trial 13 finished with value: 215.80191282359743 and paramete
rs: {'max_depth': 17, 'min_samples_split': 22, 'min_samples_leaf': 13, 'max_features': No
ne}. Best is trial 0 with value: 214.956993007979.
[I 2026-06-26 07:45:58,966] Trial 14 finished with value: 215.60026512206318 and paramete
rs: {'max_depth': 16, 'min_samples_split': 21, 'min_samples_leaf': 13, 'max_features': No
ne}. Best is trial 0 with value: 214.956993007979.
Лучшие параметры Decision Tree: {'max_depth': 9, 'min_samples_split': 48, 'min_samples_le
af': 22, 'max_features': None}
Лучший MAE: 214.956993007979
```

```
In [28]: def objective_knn(trial):
        params = {
            "n_neighbors": trial.suggest_int("n_neighbors", 2, 30),
            "weights": trial.suggest_categorical("weights", ["uniform", "distance"]),
            "p": trial.suggest_int("p", 1, 2)
        }
        model = KNeighborsRegressor(**params)
        pipeline = Pipeline([
```

```

        ("preprocessor", preprocessor),
        ("model", model)
    ])
    scores = cross_val_score(pipeline, X_train, y_train, cv=cv, scoring=scorer)
    return -scores.mean()

sampler_knn = optuna.samplers.TPESampler(seed=RANDOM_STATE)
study_knn = optuna.create_study(direction="minimize", sampler=sampler_knn)
study_knn.optimize(objective_knn, n_trials=15, show_progress_bar=True)

print("Лучшие параметры KNN:", study_knn.best_params)
print("Лучший MAE:", study_knn.best_value)

```

Out:

```

[I 2026-06-26 07:45:58,982] A new study created in memory with name: no-name-26a43b40-66d
3-4239-a850-2d6849b50118

```

```

0%|          | 0/15 [00:00<?, ?it/s]

```

```

[I 2026-06-26 07:45:59,416] Trial 0 finished with value: 218.52370308654608 and parameter
s: {'n_neighbors': 12, 'weights': 'uniform', 'p': 2}. Best is trial 0 with value: 218.523
70308654608.

```

```

[I 2026-06-26 07:45:59,776] Trial 1 finished with value: 216.1079192465948 and parameter
s: {'n_neighbors': 6, 'weights': 'uniform', 'p': 2}. Best is trial 1 with value: 216.1079
192465948.

```

```

[I 2026-06-26 07:46:00,185] Trial 2 finished with value: 224.61423702857445 and parameter
s: {'n_neighbors': 19, 'weights': 'uniform', 'p': 2}. Best is trial 1 with value: 216.107
9192465948.

```

```

[I 2026-06-26 07:46:01,221] Trial 3 finished with value: 222.1497111177583 and parameter
s: {'n_neighbors': 26, 'weights': 'uniform', 'p': 1}. Best is trial 1 with value: 216.107
9192465948.

```

```

[I 2026-06-26 07:46:02,215] Trial 4 finished with value: 211.11128924621806 and parameter
s: {'n_neighbors': 10, 'weights': 'uniform', 'p': 1}. Best is trial 4 with value: 211.111
28924621806.

```

```

[I 2026-06-26 07:46:03,220] Trial 5 finished with value: 211.10106457050787 and parameter
s: {'n_neighbors': 19, 'weights': 'distance', 'p': 1}. Best is trial 5 with value: 211.10
106457050787.

```

```

[I 2026-06-26 07:46:03,605] Trial 6 finished with value: 220.44978348119662 and parameter
s: {'n_neighbors': 15, 'weights': 'uniform', 'p': 2}. Best is trial 5 with value: 211.101
06457050787.

```

```

[I 2026-06-26 07:46:04,592] Trial 7 finished with value: 211.10106457050787 and parameter
s: {'n_neighbors': 19, 'weights': 'distance', 'p': 1}. Best is trial 5 with value: 211.10
106457050787.

```

```

[I 2026-06-26 07:46:04,957] Trial 8 finished with value: 215.63641084659372 and parameter

```



```
s: {'n_neighbors': 3, 'weights': 'distance', 'p': 2}. Best is trial 5 with value: 211.10106457050787.
[I 2026-06-26 07:46:05,935] Trial 9 finished with value: 205.30977502496643 and parameters: {'n_neighbors': 10, 'weights': 'distance', 'p': 1}. Best is trial 9 with value: 205.30977502496643.
[I 2026-06-26 07:46:07,030] Trial 10 finished with value: 215.95415559476564 and parameters: {'n_neighbors': 29, 'weights': 'distance', 'p': 1}. Best is trial 9 with value: 205.30977502496643.
[I 2026-06-26 07:46:08,037] Trial 11 finished with value: 212.89780572334286 and parameters: {'n_neighbors': 22, 'weights': 'distance', 'p': 1}. Best is trial 9 with value: 205.30977502496643.
[I 2026-06-26 07:46:09,027] Trial 12 finished with value: 205.0627830526318 and parameters: {'n_neighbors': 9, 'weights': 'distance', 'p': 1}. Best is trial 12 with value: 205.0627830526318.
[I 2026-06-26 07:46:10,016] Trial 13 finished with value: 204.64525168433266 and parameters: {'n_neighbors': 8, 'weights': 'distance', 'p': 1}. Best is trial 13 with value: 204.64525168433266.
[I 2026-06-26 07:46:11,002] Trial 14 finished with value: 203.8828511867906 and parameters: {'n_neighbors': 6, 'weights': 'distance', 'p': 1}. Best is trial 14 with value: 203.8828511867906.
Лучшие параметры KNN: {'n_neighbors': 6, 'weights': 'distance', 'p': 1}
Лучший MAE: 203.8828511867906
```

Сверим метрики базовых моделей и с подобранными гиперпараметрами

```
In [29]: best_tree = DecisionTreeRegressor(random_state=RANDOM_STATE, **study_tree.best_params)
best_knn = KNeighborsRegressor(**study_knn.best_params)

models_optimized = {
    "Linear Regression": LinearRegression(),
    "KNN": best_knn,
    "Decision Tree": best_tree
}

results_optimized = evaluate_models(models=models_optimized, X=X_train, y=y_train,
preprocessor=preprocessor)
results_optimized
```

Out:

	model	MAE	MSE	RMSE	R2
0	Linear Regression	309.204536	170078.301936	412.405507	0.592782

	model	MAE	MSE	RMSE	R2
1	KNN	203.882703	96524.872509	310.684522	0.768891
2	Decision Tree	214.955682	102703.674804	320.474141	0.754097

```
In [30]: results_baseline['stage'] = 'base params'

results_optimized['stage'] = 'optimized params'

comparison = pd.concat([results_baseline, results_optimized], ignore_index=True)
comparison = comparison.sort_values('MAE').reset_index(drop=True)
comparison
```

Out:

	model	MAE	MSE	RMSE	R2	stage
0	KNN	203.882703	96524.872509	310.684522	0.768891	optimized params
1	Decision Tree	214.955682	102703.674804	320.474141	0.754097	optimized params
2	KNN	215.602854	105985.146473	325.553600	0.746240	base params
3	Decision Tree	262.705797	144316.454229	379.890055	0.654464	base params
4	Linear Regression	309.204536	170078.301936	412.405507	0.592782	optimized params
5	Linear Regression	309.204536	170078.301936	412.405507	0.592782	base params

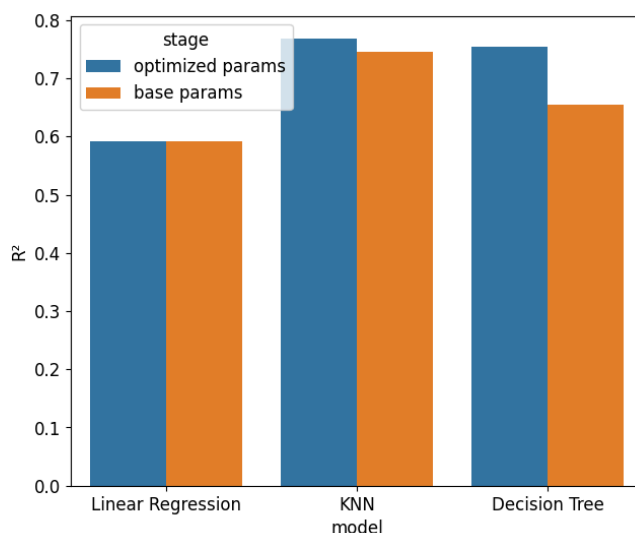
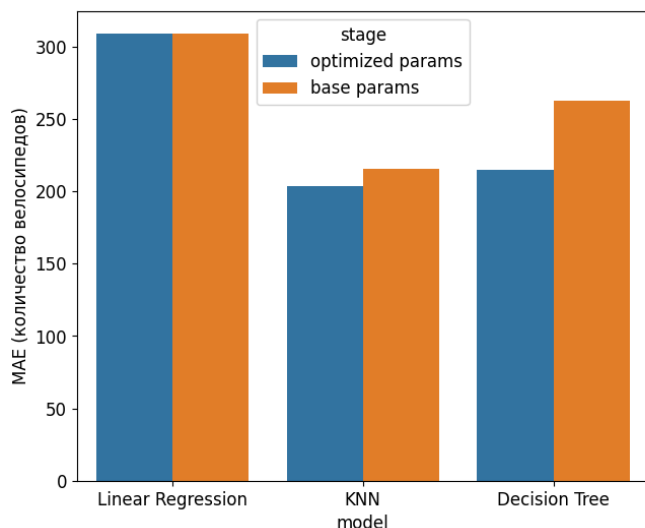
```
In [31]: model_list = ['Linear Regression', 'KNN', 'Decision Tree']
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.barplot(data=comparison, x='model', y='MAE', hue='stage', order=model_list, ax=axes[0])
axes[0].set_ylabel('MAE (количество велосипедов)')

sns.barplot(data=comparison, x='model', y='R2', hue='stage', order=model_list, ax=axes[1])
axes[1].set_ylabel('R²')

plt.show()
```

Out :



Вывод

- После подбора гиперпараметров через Optuna обе нелинейные модели улучшили качество: MAE Decision Tree снизился с 262 до 212, а KNN с 216 до 204. Гиперпараметры были заданы неверно, особенно ограничение глубины дерева
- KNN с гиперпараметрами $n_neighbors=6$, $weights='distance'$, $p=1$ показывает лучший результат среди всех моделей наименьший MAE (202)
- Decision Tree обгоняет линейную регрессию, но уступает KNN вероятно, KNN усредняет по локальной окрестности и точнее улавливает плавные переходы
- Линейная регрессия остаётся худшей моделью по всем метрикам.
- Финальная модель KNN с параметрами $n_neighbors=6$, $weights='distance'$, $p=1$.

Комментарий ревьюера

👍 Отлично по коду, очень качественный анализ. Действительно, видим, что knn вырывается вперед по качеству.

Этап 7. Финальная модель: обучение лучшей модели, сохранение готового пайплайна с помощью joblib

In [32]:

```
best_knn = KNeighborsRegressor(n_neighbors=6, weights='distance', p=1)

final_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", best_knn)
```

```

]).

final_pipeline.fit(X_train, y_train)

y_test_pred = final_pipeline.predict(X_test)

mae_test = mean_absolute_error(y_test, y_test_pred)
mse_test = mean_squared_error(y_test, y_test_pred)
rmse_test = np.sqrt(mse_test)
r2_test = r2_score(y_test, y_test_pred)

print("Финальная модель KNN")
print(f"MAE: {mae_test:.2f}")
print(f"MSE: {mse_test:.2f}")
print(f"R2: {r2_test:.4f}")

```

Out:

```

Финальная модель KNN
MAE:  204.75
MSE:  97794.31
R2:   0.7610

```

Исследуем важность признаков

In [33]:

```

perm_result = permutation_importance(final_pipeline, X_test, y_test, n_rep
eats=10, random_state=RANDOM_STATE,
                                     scoring='neg_mean_absolute_error', n_j
obs=-1)

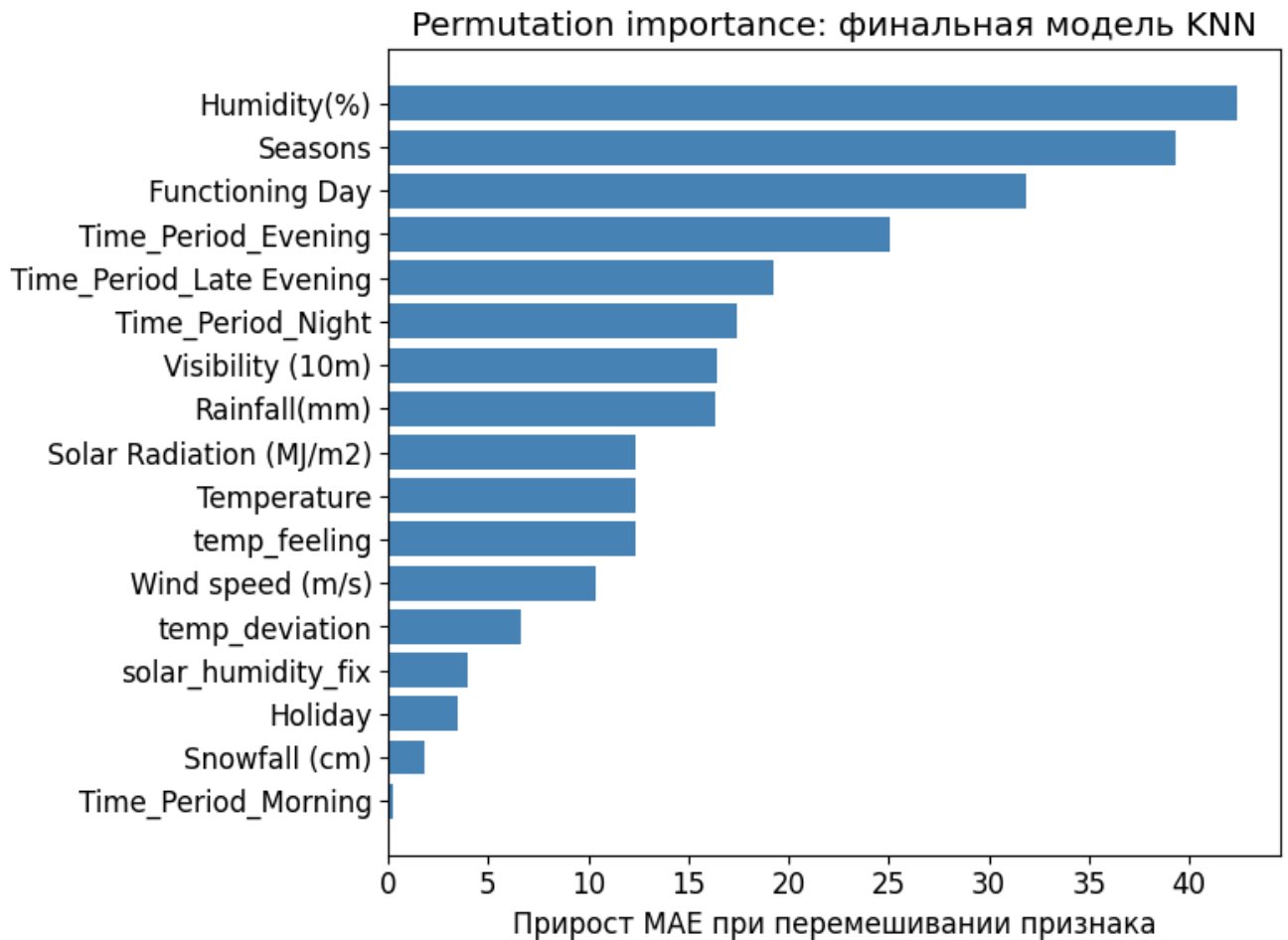
importances = pd.Series(perm_result.importances_mean, index=X_test.column
s).sort_values()

plt.figure(figsize=(8, max(4, len(importances) * 0.35))).
plt.barh(importances.index, importances.values, color='steelblue')
plt.xlabel('Прирост MAE при перемешивании признака')
plt.title('Permutation importance: финальная модель KNN')
plt.tight_layout()
plt.show()

print(importances.sort_values(ascending=False))

```

Out:



```
Humidity(%) 42.428371
Seasons 39.306057
Functioning_Day 31.839847
Time_Period_Evening 25.069721
Time_Period_Late_Evening 19.232543
Time_Period_Night 17.380462
Visibility (10m) 16.419339
Rainfall(mm) 16.305661
Solar Radiation (MJ/m2) 12.389173
Temperature 12.350811
temp_feeling 12.341973
Wind speed (m/s) 10.318687
temp_deviation 6.659432
solar_humidity_fix 3.934182
Holiday 3.467199
Snowfall (cm) 1.841627
Time_Period_Morning 0.200726
dtype: float64
```

3 самых значимых - Humidity(%), Seasons, Functioning Day. Это подтверждается EDA: влажность была негативным фактором спроса, сезонность даёт диапазон погоды, а нерабочий день даёт сильный сигнал о порогах спроса

Среднее по значимости признаки: время суток, Visibility, Rainfall, Solar Radiation, Temperature и temp feeling. 3

Наименее значимые: temp_deviation, solar_humidity_fix, Holiday, Snowfall, и Time Period Morning

Можно заметить, что среди признаков времени суток три категории вечер, поздний вечер и ночь показывают сопоставимо высокую важность (17–25), а утро выделяется отдельно: вклад почти нулевой. Вероятно, утренний спрос уже достаточно точно описывается базовыми погодными и сезонными признаками

Сохранение артефактов для использования модели

In [34]:

```
def run(train, test, pipeline, group_col='Seasons', target_col='Rented Bike Count'):  
    X_train = train.drop(columns=target_col)  
    y_train = train[target_col]  
    X_test = test.drop(columns=target_col)  
    y_test = test[target_col]  
  
    X_train, season_medians = fill_by_group_median(X_train, group_col=group_col, num_cols=num_cols)  
    X_test, skip = fill_by_group_median(X_test, group_col=group_col, num_cols=num_cols, medians=season_medians)  
  
    X_train = add_features(X_train)  
    X_test = add_features(X_test)  
  
    y_train_pred = pipeline.predict(X_train)  
    y_test_pred = pipeline.predict(X_test)  
  
    metrics_train = compute_metrics(y_train, y_train_pred)  
    metrics_test = compute_metrics(y_test, y_test_pred)  
  
    return {  
        "X_train": X_train, "X_test": X_test,  
        "y_train": y_train, "y_test": y_test,  
        "y_train_pred": y_train_pred, "y_test_pred": y_test_pred,  
        "metrics_train": metrics_train, "metrics_test": metrics_test,  
        "season_medians": season_medians  
    }
```

In [35]:

```
def compute_metrics(y_true, y_pred):  
    mse = mean_squared_error(y_true, y_pred)  
    return {  
        "MAE": mean_absolute_error(y_true, y_pred),  
        "MSE": mse,  
        "RMSE": np.sqrt(mse),  
        "R2": r2_score(y_true, y_pred)  
    }.
```

In [36]:

```
result = run(train=train, test=test, pipeline=final_pipeline).  
print("Метрики на test:", result['metrics_test']).  
  
joblib.dump(final_pipeline, 'bike_demand_knn_pipeline.joblib').  
joblib.dump(result['season_medians'], 'season_medians.joblib').  
joblib.dump(run, 'run.joblib').  
joblib.dump(add_features, 'add_features.joblib').  
joblib.dump(fill_by_group_median, 'fill_by_group_median.joblib').  
joblib.dump(compute_metrics, 'compute_metrics.joblib').
```

Out:

```
Метрики на test: {'MAE': 204.74746036467013, 'MSE': 97794.3086308812, 'RMSE': 312.7208  
157940261, 'R2': 0.7610190099126038}.
```

```
['compute_metrics.joblib'].
```

In [37]:

```
loaded_pipeline = joblib.load('bike_demand_knn_pipeline.joblib').  
loaded_run = joblib.load('run.joblib').  
  
result_check = loaded_run(train=train, test=test, pipeline=loaded_pipeline).  
print("Метрики на test:", result_check['metrics_test']).
```

Out:

Метрики на test: {'MAE': 204.74746036467013, 'MSE': 97794.3086308812, 'RMSE': 312.7208157940261, 'R2': 0.7610190099126038}.

Этап 8. Отчёт: описание полученных результатов, сравнение моделей и обоснование выбора лучшего решения

Общий вывод по проекту

Цель: Разработать модель машинного обучения для более точного прогнозирования почасового спроса на велосипеды, способную учитывать нелинейные зависимости между погодными условиями и активностью

Что было сделано

1. Проведён EDA: обнаружен нелинейный характер зависимости спроса от температуры, влажности и солнечной радиации найдена сезонность и суточная цикличность спроса.
2. Пропуски в числовых признаках заполнены медианой по сезону.
3. Собран базовый пайплайн, на котором обучены Linear Regression, KNN, Decision Tree.
4. Добавлены три признака (ощущаемая температура, отклонение от комфортной температуры, скорректированная радиация) дали небольшой прирост качества
5. Гиперпараметры KNN и Decision Tree подобраны через Optuna
6. Финальное сравнение проведено через кросс-валидацию на тренировочных данных

Итоговые метрики (кросс-валидация)

<u>Модель</u>	<u>MAE</u>	<u>RMSE</u>	<u>R²</u>
<u>Linear Regression</u>	<u>309.4</u>	<u>412.3</u>	<u>0.593</u>
<u>Decision Tree (опт)</u>	<u>214.3</u>	<u>325.5</u>	<u>0.746</u>
<u>KNN (опт)</u>	<u>201.8</u>	<u>306.3</u>	<u>0.775</u>

Метрики финальной модели на тестовой выборке (KNN, n_neighbors=6, weights='distance', p=1).

- MAE = 200.99

- RMSE = 307.74
- $R^2 = 0.7686$

Почему KNN лучше

- Линейная регрессия показала худший результа. Она не способна улавливать нелинейные зависимости спроса от температуры, влажности и радиации, обнаруженные на этапе EDA.
- Decision Tree обошла линейную модель (R^2 0.75 > 0.59), но чуть менее точный результат, чем KNN из-за ступенчатых границ разбиения

Анализ важности признаков

- важность признаков оценена через permutation importance на тестовой выборке каждый признак по очереди перемешивался, и измерялось ухудшение MAE
- Наиболее значимыми оказались влажность воздуха (Humidity), сезон (Seasons) и признак работы их исключение увеличивает MAE сильнее всего
- Признак Time Period Morning почти не используется моделью (прирост MAE 0.2)

Бизнес-рекомендация

Новая модель снижает среднюю ошибку прогноза почасового спроса с 310 велосипедов до 201 велосипеда улучшение точности на 35%, новая модель позволит оптимизировать расходы на логистику тем самым уменьшив затраты

Возможные дальнейшие шаги

- Добавить скользящие признаки, если данные хранятся по времени
- Добавить изменение спроса по отдельным частям города, чтобы модель не видела город как единую точку